

RURALNET TRAINING COLLEGE



HTML, CSS, JAVASCRIPT

Mr. Boaz Vidal

Copyright

Copyright © 2026 [Mr. Boaz Ochieng]

All rights reserved.

No part of this book may be reproduced, distributed, stored in a retrieval system, or transmitted in any form or by any means, whether electronic, mechanical, photocopying, recording, or otherwise, without prior written permission from the author, except for brief quotations used for educational, review, or reference purposes as permitted by applicable copyright law.

This book has been developed for educational and instructional purposes. It contains explanations, code examples, practical activities, exercises, and projects designed to develop learners' knowledge and skills in web development using HTML, CSS, and JavaScript.

Although every effort has been made to ensure the accuracy of the information and code examples, the author does not guarantee that every example will function identically across all browsers, devices, or software versions. Readers are encouraged to test, modify, and improve the examples as part of their learning process.

First Edition: 2026

Author: [Mr. Boaz Ochieng]

Publisher/Institution: [Ruralnet Training College]

Contact: [ruralnet2016@gmail.com, +254700886058]

ISBN:

Dedication

I dedicate this book to all learners, educators, aspiring programmers, and technology enthusiasts who believe in the power of knowledge and innovation.

To every student who is willing to learn, experiment, make mistakes, and try again, may this book serve as a valuable companion throughout your journey into web development.

I also dedicate this work to educators and trainers who continue to inspire creativity, promote digital literacy, and equip learners with the practical skills needed to participate meaningfully in the digital world.

Acknowledgement

The development of this book has been inspired by the growing importance of web technologies in education, business, communication, and modern society.

I sincerely acknowledge the contribution of educators, software developers, technical writers, and the wider programming community whose knowledge, experience, and shared resources continue to make computer programming more accessible to learners around the world.

Special appreciation goes to all teachers and trainers who dedicate their time to helping learners understand programming concepts and apply them to real-life situations.

I also appreciate the learners whose curiosity, creativity, and determination reinforce the need for practical, accessible, and structured learning materials.

Finally, I extend my gratitude to everyone who has contributed directly or indirectly to the preparation of this book. Their encouragement and commitment to learning have helped shape this educational resource.

Preamble

The rapid advancement of information and communication technology has transformed the way people learn, communicate, conduct business, and access information. Websites and web-based applications have become essential tools for organisations, educational institutions, businesses, and individuals.

As a result, web development has emerged as an important digital skill, providing opportunities for creativity, problem-solving, entrepreneurship, and professional growth.

This book, A Practical Guide to HTML, CSS and JavaScript, has been developed to provide learners with a structured and practical introduction to the fundamental technologies used in building modern websites. It combines theoretical explanations with practical demonstrations, exercises, and projects to bridge the gap between conceptual understanding and actual implementation.

The material is organised to guide learners from basic concepts to more advanced applications at a pace that encourages understanding and independent practice. Each technology is introduced according to its role in web development, enabling learners to appreciate how HTML structures content, CSS controls presentation, and JavaScript introduces interaction and dynamic behaviour.

The book is intended to serve as both a classroom reference and a self-study resource. It encourages learners not merely to memorise code, but to understand its purpose, apply it appropriately, identify errors, and develop their own solutions.

It is my hope that this book will inspire readers to explore web technologies with confidence and use their acquired knowledge to create meaningful, functional, and innovative digital solutions.

Introduction

Web development is the process of creating, designing, and maintaining websites and web applications. It involves several technologies that work together to produce web pages that are structured, visually appealing, interactive, and accessible through web browsers.

This book focuses on three foundational technologies: HTML, CSS, and JavaScript.

HTML (HyperText Markup Language) provides the structure of a web page. It defines elements such as headings, paragraphs, images, hyperlinks, lists, tables, forms, and other content.

CSS (Cascading Style Sheets) controls the appearance and layout of web pages. It allows developers to apply colours, backgrounds, fonts, spacing, borders, shadows, animations, responsive layouts, and other visual effects.

JavaScript introduces behaviour and interactivity. It enables web pages to respond to user actions, validate forms, manipulate page content, perform calculations, display messages, and implement dynamic features.

When used together, these technologies provide the foundation for developing a wide range of websites, from simple personal pages to interactive educational platforms and business applications.

This book adopts a practical, learner-centred approach. It combines theory with code examples, guided practical activities, review exercises, and projects that encourage learners to apply their knowledge.

Readers are encouraged to practise each concept in a code editor, run their work in a web browser, observe the results, and modify the examples to explore alternative solutions. The emphasis is on understanding, experimentation, creativity, and the gradual development of independent programming skills.

By the end of the learning process, readers should have a foundation for creating their own web pages and combining HTML, CSS, and JavaScript to develop interactive web-based solutions.

Purpose of the Book

The purpose of this book is to provide a comprehensive and accessible learning resource that introduces learners to web development through a combination of theory and practical application.

Specifically, the book seeks to:

1. Introduce the fundamental concepts and principles of web development.
2. Develop learners' understanding of HTML elements, attributes, and document structure.
3. Equip learners with the skills to style web pages using CSS.
4. Introduce JavaScript programming concepts and their application in interactive web pages.
5. Promote logical thinking, creativity, debugging, and problem-solving.
6. Provide structured exercises for reinforcing knowledge and evaluating understanding.
7. Guide learners in developing practical projects that demonstrate their acquired skills.
8. Encourage independent learning and continued exploration of web technologies.

Learning Objectives

Upon successful completion of this book, learners should be able to:

- Explain the meaning and importance of web development.
- Differentiate between HTML, CSS, and JavaScript.
- Create and organise valid HTML documents.
- Insert text, images, hyperlinks, tables, lists, and forms into web pages.
- Apply CSS selectors, properties, values, and styling rules.
- Design page layouts using appropriate spacing, positioning, and layout techniques.
- Create responsive and visually appealing web interfaces.
- Explain JavaScript variables, data types, operators, conditions, loops, functions, and events.
- Use JavaScript to manipulate HTML elements and respond to user interactions.
- Identify and correct common errors in HTML, CSS, and JavaScript.
- Integrate the three technologies into functional websites.
- Plan, develop, test, and present small web development projects.

Target Audience

This book is intended for a broad range of learners and individuals interested in acquiring practical web development skills.

The primary target audience includes:

- Beginners with little or no prior programming experience.
- Students undertaking computer studies, ICT, or related courses.
- Trainees in computer colleges and technical training institutions.
- Teachers and trainers seeking structured instructional material.
- Individuals interested in creating personal, educational, or business websites.
- Aspiring web developers seeking a foundation for further study.
- Self-directed learners who prefer learning through examples and practical projects.

Although the book is designed to support beginners, readers who already possess some programming knowledge may also use it as a reference for revision and practical exercises.

How to Use This Book

To obtain maximum benefit from this book, learners are encouraged to follow the chapters sequentially, especially if they are new to programming.

The book combines several learning approaches, each serving a specific purpose.

Theory

Read the explanations carefully to understand the meaning, purpose, syntax, and application of each concept before attempting the practical activities.

Practical Lessons

Type the code examples into a code editor instead of merely reading them. Save the files, open them in a browser, and observe the output.

Exercises

Attempt the questions independently to assess your understanding. Where possible, explain your answers and write the required code without referring immediately to the examples.

Projects

Apply several concepts together to create complete solutions. Plan your work, organise your files, test the results, correct errors, and improve the final design.

Recommended learning procedure

1. Read the theory.
2. Study the code examples.
3. Practise the examples on a computer.
4. Complete the exercises.
5. Attempt the related project.
6. Test your work and correct errors.
7. Review the chapter before proceeding.

Learners should maintain an organized folder for their work and save their projects regularly. Consistent practice is more effective than memorizing code without understanding how it works.

Requirements for Learning

Learners will benefit from having access to the following resources:

- A desktop computer or laptop.
- A modern web browser such as Google Chrome, Microsoft Edge, Firefox, or another standards-compliant browser.
- A code editor such as Visual Studio Code or a suitable alternative.
- Basic computer operation and file management skills.
- A dedicated folder for saving HTML, CSS, JavaScript, images, and project files.
- Regular access to practical activities and sufficient time for independent practice.

Most introductory HTML, CSS, and JavaScript activities can be completed without an internet connection once the necessary software and files are available. However, internet access may be required for external libraries, online references, fonts, or other web-based resources used in particular exercises.

Organisation of the Book

The book is organised around the three core technologies of front-end web development, supported by practical exercises and projects.

Section	Main focus
Preliminary Pages	Background, purpose, objectives, and guidance for learners
HTML	Document structure, elements, attributes, text, links, media, tables, and forms
CSS	Styling, selectors, colours, typography, spacing, layouts, effects, and responsive design
JavaScript	Syntax, variables, operators, conditions, loops, functions, events, and DOM manipulation
Exercises	Knowledge checks, coding tasks, and problem-solving activities
Practical Projects	Integrated website development and application of learned concepts
Revision and Reference	Consolidation of key ideas and support for further learning

Note: Adjust the section descriptions to match your actual chapters and the order in which the topics appear in your book.

Teaching and Learning Methodology

This book adopts a learner-centred, practical approach that combines explanation, demonstration, guided practice, independent work, and project-based learning.

The methodology is based on the principle that programming skills develop through repeated application rather than theoretical study alone.

The learning process incorporates the following methods:

- Explanation: Concepts are introduced and clarified using definitions and descriptions.
- Demonstration: Code examples illustrate the application of specific programming techniques.
- Guided practice: Learners follow structured instructions to reproduce and modify examples.
- Independent practice: Exercises allow learners to test their understanding and develop confidence.
- Problem-solving: Coding tasks encourage logical reasoning and systematic debugging.
- Project-based learning: Larger activities require learners to combine multiple concepts into practical solutions.
- Evaluation and reflection: Learners review their results, identify weaknesses, and improve their work.

Teachers and trainers may use the material for classroom demonstrations, individual assignments, group activities, practical assessments, or guided project development.

Assessment and Evaluation

Assessment is an important component of learning web development because it helps determine whether learners can understand concepts and apply them correctly.

The assessment activities in this book may include:

1. Oral and written questions to evaluate conceptual understanding.
2. Short exercises to assess knowledge of syntax, terminology, and programming principles.
3. Practical tasks to measure the ability to write and execute code.
4. Debugging activities to evaluate error identification and correction.
5. Individual projects to assess independent application of skills.
6. Integrated projects to evaluate the ability to combine HTML, CSS, and JavaScript.
7. Project presentations to encourage learners to explain their design decisions and demonstrate their work.

When evaluating practical work, consideration should be given to correctness, functionality, code organisation, readability, design, usability, and the learner's ability to explain how the solution works.

Note to the Learner

Learning to program requires patience, curiosity, discipline, and consistent practice. At first, some concepts may appear difficult, and your code may not always produce the expected results. These experiences are a normal part of the learning process.

Do not be discouraged by errors. Instead, examine them carefully, identify their causes, and attempt to correct them systematically. Each mistake provides an opportunity to develop a deeper understanding of how programming works.

As you progress through this book, avoid copying code without understanding its purpose. Whenever possible, modify the examples, change the values, experiment with different styles, and create your own variations.

You are encouraged to think creatively and approach every exercise as an opportunity to develop a practical solution. The projects should not be treated merely as tasks to complete, but as opportunities to demonstrate initiative, originality, and technical competence.

Remember that proficiency in web development comes through continuous practice. The more you write, test, debug, and improve your code, the more confident and capable you will become.

Note to the Instructor

This book may be used as a teaching aid for introductory web development lessons in schools, colleges, training centres, and other learning environments.

Instructors are encouraged to adapt the pace of instruction to learners' abilities and available resources. Lessons should balance conceptual explanations with live demonstrations and supervised practical work.

Where possible, learners should be given opportunities to solve problems independently, collaborate on selected projects, explain their code, and present completed websites.

Instructors may supplement the material with additional examples, current industry practices, accessibility guidelines, responsive design techniques, and appropriate assessment tasks.

The overall objective should be to develop learners who understand the principles behind the code and can apply their knowledge to new situations rather than merely reproduce familiar examples.

List of Abbreviations and Technical Terms

Term	Meaning
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
WWW	World Wide Web
URL	Uniform Resource Locator
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
DOM	Document Object Model
IDE	Integrated Development Environment
UI	User Interface
UX	User Experience
RGB	Red, Green, Blue
API	Application Programming Interface
SEO	Search Engine Optimisation
W3C	World Wide Web Consortium

General Summary of the Book

Web development is an essential digital skill that enables individuals and organisations to create websites and interactive web applications. HTML, CSS, and JavaScript form the foundation of this process, with each technology performing a distinct but complementary role.

HTML provides the structure and meaning of web content. CSS controls its presentation, layout, and visual appearance. JavaScript adds behaviour and interactivity, allowing web pages to respond to user input and perform dynamic operations.

Understanding these technologies individually is important, but their true potential is realised when they are integrated into complete web solutions.

Through theoretical explanations, practical demonstrations, exercises, and projects, this book aims to develop learners' understanding and practical competence. It encourages systematic thinking, independent experimentation, effective debugging, and creative problem-solving.

Ultimately, successful learning should enable readers to move beyond understanding individual code statements to designing, developing, testing, and improving functional websites. The knowledge acquired through this book can also provide a foundation for exploring more advanced areas of web development and related technologies.

Conclusion

The ability to develop websites is increasingly valuable in a world where digital communication, online services, and technology-driven solutions continue to expand.

A sound understanding of HTML, CSS, and JavaScript provides learners with the essential tools required to begin creating web-based solutions. However, technical knowledge alone is not sufficient; effective web development also requires creativity, logical reasoning, attention to detail, continuous practice, and a willingness to learn.

This book provides a starting point for developing these abilities through structured learning and practical application. By completing the exercises and projects, learners are encouraged to transform theoretical knowledge into demonstrable skills.

The journey does not end with the final chapter. Rather, it marks the beginning of continued exploration, experimentation, and professional development in the field of web technologies.

Hypertext Markup Language (HTML)

Images, Links, Multimedia, Text Formatting, Lists, Tables, Buttons and Forms

---

1. HTML Links — The `<a>` Tag

The anchor tag, `<a>`, is used to create hyperlinks. A hyperlink allows a user to move from one webpage to another, open a document, send an email, or visit an external website.

Syntax

```
<a href="https://www.google.com">Visit Google</a>
```

The `href` Attribute

The `href` attribute specifies the destination of the link.

Example 1: Linking to an external website

```
<a href="https://www.google.com">Google</a>
```

Example 2: Linking to another page in the same folder

```
<a href="about.html">About Us</a>
```

Example 3: Opening an email application

```
<a href="mailto:info@example.com">Send Email</a>
```

Example 4: Opening a link in a new tab

```
<a href="https://www.google.com" target="_blank">
    Open Google
</a>
```

Note: For added security when using `target="_blank"`, you can include `rel="noopener noreferrer"`.

---

2. Images — The `<img>` Tag

The `<img>` tag is used to display an image on a webpage. It is a void element, meaning it does not have a closing tag.

Syntax

```
<img src="photo.jpg" alt="A beautiful landscape">
```

Important Image Attributes

- `src` — Specifies the path or location of the image file.
- `alt` — Provides alternative text describing the image. This helps screen-reader users and appears when the image cannot load.
- `height` — Specifies the image height.
- `width` — Specifies the image width.

Example

```
<img src="student.jpg"
     alt="A student using a computer"
     width="300"
     height="200">
```

The image will be displayed at a width of 300 pixels and a height of 200 pixels.

Important: Setting both width and height to values that do not match the image's original proportions can stretch or distort it.

2.1 Surrounding an Image with a Link

An image can be made clickable by placing it inside an anchor tag.

```
<a href="https://www.google.com">
    <img src="google.jpg"
         alt="Visit Google"
         width="200">
</a>
```

When the user clicks the image, the browser opens the specified destination.

2.2 Adding an Animated GIF

GIF stands for Graphics Interchange Format. It supports simple animations and can be displayed using the `<img>` tag.

```
<img src="welcome.gif"
     alt="Welcome animation"
     width="300">
```

The GIF animation usually plays automatically when displayed.

2.3 Relative File Paths

A relative file path describes the location of a file in relation to the current HTML file.

Consider this folder structure:

```
MyWebsite/
│
├── index.html
├── about.html
│
├── images/
│   ├── logo.png
│   └── student.jpg
│
└── media/
    └── welcome.gif
```

Examples of relative paths:

```
<!-- File in the same folder -->
<img src="logo.png" alt="Logo">

<!-- File inside the images folder -->
<img src="images/logo.png" alt="Website logo">

<!-- File inside the media folder -->
<img src="media/welcome.gif" alt="Welcome animation">

<!-- File in the parent folder -->
<img src="../logo.png" alt="Logo">
```

The last example works when the HTML file is inside a subfolder and the image is in its parent folder.

Remember: File and folder names must match their actual names. Incorrect paths cause images and other media to fail to load.

---

3. Self-Closing and Void Elements

In HTML, some elements do not need closing tags. These are called **void elements**.

Examples include:

```
<img src="photo.jpg" alt="Photo">
<br>
<hr>
<input type="text">
<link rel="icon" href="favicon.ico">
<meta charset="UTF-8">
```

In HTML5, the slash in `<img />` is optional. Both `<img>` and `<img />` are valid HTML syntax.

However, elements such as `<p>`, `<div>`, `<audio>`, and `<video>` require closing tags.

Correct:

```
<p>This is a paragraph.</p>
```

Incorrect:

```
<p>This is a paragraph.
```

---

4. Audio in HTML

The `<audio>` tag is used to play sound files on a webpage.

Example

```
<audio controls>
    <source src="music.mp3" type="audio/mpeg">
    Your browser does not support audio playback.
</audio>
```

Common Audio Attributes

- `controls` — Displays play, pause, volume and other playback controls.
- `autoplay` — Requests that playback start automatically.
- `muted` — Starts the audio muted.
- `loop` — Repeats the audio when it finishes.
- `src` — Specifies the audio file location when used directly on the audio element.

Example with Multiple Attributes

```
<audio controls autoplay muted loop>
    <source src="music.mp3" type="audio/mpeg">
</audio>
```

The browser requests autoplay, but the audio starts muted. Browser autoplay policies may prevent playback in some circumstances.

Common Audio File Types

- MP3 — `audio/mpeg`
- WAV — `audio/wav`
- OGG — `audio/ogg`

The `<source>` element allows the browser to select a supported file format.

---

5. Video in HTML

The `<video>` tag is used to embed video content in a webpage.

Example

```
<video width="500" controls>
    <source src="lesson.mp4" type="video/mp4">
    Your browser does not support video playback.
</video>
```

Common Video Attributes

- `src` — Specifies the video file when used directly on the `<video>` tag.
- `width` — Sets the video width.
- `height` — Sets the video height.
- `controls` — Displays playback controls.
- `autoplay` — Requests automatic playback.
- `muted` — Mutes the video audio.
- `loop` — Repeats the video.
- `poster` — Specifies an image displayed before the video starts.

Example with Autoplay and Loop

```
<video width="500" autoplay muted loop controls>
    <source src="lesson.mp4" type="video/mp4">
</video>
```

Example with a Poster Image

```
<video width="500" controls poster="thumbnail.jpg">
    <source src="lesson.mp4" type="video/mp4">
</video>
```

Common video formats include MP4 (`video/mp4`), WebM (`video/webm`) and Ogg (`video/ogg`). Browser support also depends on the codecs used inside the file.

---

6. Adding a Favicon

A favicon is a small icon associated with a website. It commonly appears in the browser tab, bookmarks and other browser interfaces.

Place the favicon file in the same folder as your HTML file, or specify the correct relative path.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Website</title>

    <link rel="icon" type="image/jpeg" href="favicon.jpg">
</head>
<body>
    <h1>Welcome to My Website</h1>
</body>
</html>
```

Using a PNG Favicon

```
<link rel="icon" type="image/png" href="images/favicon.png">
```

Using an ICO Favicon

```
<link rel="icon" type="image/x-icon" href="favicon.ico">
```

Note: The `type` attribute should match the actual image format. A JPG image should not be labelled as PNG or ICO.

---

7. Text Formatting Tags

HTML provides elements for formatting or giving special meaning to text.

Tag	Purpose	Example
`<b>`	Draws attention using bold text	`<b>Hello</b>`

`<i>`	Displays text in italics	`<i>Hello</i>`
`<u>`	Underlines text	`<u>Hello</u>`
`<del>`	Indicates deleted text	`<del>500</del>`
`<big>`	Historically enlarged text; obsolete in HTML5	Avoid in new code
`<small>`	Displays smaller text	`<small>Terms apply</small>`
`<sub>`	Displays subscript text	`H<sub>2</sub>O`
`<sup>`	Displays superscript text	`x<sup>2</sup>`
`<tt>`	Historically displayed monospaced text; obsolete in HTML5	Use `<code>` instead
`<mark>`	Highlights text	`<mark>Important</mark>`
`<strong>`	Indicates strong importance	`<strong>Warning!</strong>`
`<em>`	Emphasises text	`<em>Please read</em>`

Example

```
<p>How are <b>you</b> today?</p>

<p>This word is <i>italic</i>.</p>

<p>This word is <u>underlined</u>.</p>

<p>Original price: <del>KSh 1,000</del></p>

<p>Water: H<sub>2</sub>O</p>

<p>Mathematics: x<sup>2</sup></p>

<p><mark>Read this carefully.</mark></p>

<p><strong>Important notice!</strong></p>

<p><small>Terms and conditions apply.</small></p>
```

---

8. The `<span>` and `<div>` Elements

These elements are used to group other HTML elements or pieces of content.

8.1 The `<span>` Element

`<span>` is an inline container. It groups a small part of text or other inline content without automatically starting a new line.

Example:

```
<p>My favourite colour is <span>green</span>.</p>
```

Another example:

```
<p>Student: <span>John</span></p>
```

8.2 The `<div>` Element

`<div>` is a block-level container by default. It groups larger sections of content and normally starts on a new line.

Example:

```
<div>
    <h2>About Our College</h2>
    <p>We offer computer training courses.</p>
</div>
```

Main Difference

- `<span>` groups inline content within a line.
- `<div>` groups block-level content into a section or container.

Neither element has a special visual appearance by default. They are often used with CSS, but they can also organize HTML content without CSS.

---

9. Lists in HTML

HTML supports three main types of lists:

1. Unordered lists
2. Ordered lists
3. Description lists

9.1 Unordered Lists — `<ul>`

An unordered list displays items using bullets by default.

```
<h3>Courses Offered</h3>

<ul>
    <li>Web Development</li>
    <li>Data Analysis</li>
    <li>Graphic Design</li>
</ul>
```

9.2 Ordered Lists — `<ol>`

An ordered list displays items in a sequence, usually numbered.

```
<h3>Steps to Register</h3>

<ol>
    <li>Visit the college.</li>
    <li>Choose a course.</li>
    <li>Complete the registration form.</li>
    <li>Pay the required fees.</li>
</ol>
```

You can change the numbering style with the `type` attribute.

```
<ol type="A">
    <li>First item</li>
    <li>Second item</li>
</ol>
```

Other common values include `1`, `a`, `A` and `i`.

9.3 Nested Lists

A nested list is a list placed inside another list item.

```
<ul>
    <li>Computer Courses
        <ul>
            <li>Computer Basics</li>
            <li>Web Development</li>
            <li>Python Programming</li>
        </ul>
    </li>

    <li>Business Courses</li>
</ul>
```

Notice that the inner `<ul>` is placed inside the relevant `<li>` element.

9.4 Description Lists — `<dl>`

A description list presents terms and their descriptions.

- `<dl>` — Defines the description list.
- `<dt>` — Defines a term or name.
- `<dd>` — Provides the description of that term.

Example:

```
<dl>
    <dt>HTML</dt>
    <dd>A markup language used to structure webpages.</dd>

    <dt>CSS</dt>
    <dd>A language used to style webpages.</dd>

    <dt>JavaScript</dt>
    <dd>A programming language used to add interactivity.</dd>

    <dt>Browser</dt>
    <dd>Software used to access and view websites.</dd>
</dl>
```

---

10. Tables in HTML

Tables organize information into rows and columns.

Important Table Tags

- `<table>` — Creates the table.
- `<tr>` — Creates a table row.
- `<th>` — Creates a header cell.
- `<td>` — Creates a standard data cell.
- `<caption>` — Adds a table title.
- `<thead>` — Groups the table header rows.
- `<tbody>` — Groups the main table body rows.

Example: Weekly Timetable

```
<table border="1">
    <caption>Weekly Class Timetable</caption>

    <tr>
        <th>Day</th>
        <th>Subject</th>
        <th>Time</th>
    </tr>

    <tr>
        <td>Monday</td>
        <td>HTML</td>
        <td>8:00 AM</td>
    </tr>

    <tr>
        <td>Tuesday</td>
        <td>Computer Basics</td>
        <td>10:00 AM</td>
    </tr>
```

```
    <tr>
        <td>Wednesday</td>
        <td>Data Analysis</td>
        <td>12:00 PM</td>
    </tr>
</table>
```

The `border="1"` attribute displays a basic border in many browsers, but it is an old presentation attribute. CSS is recommended for modern table styling.

Merging Table Cells

Use `colspan` to make a cell span multiple columns.

```
<table border="1">
    <tr>
        <th colspan="2">Student Details</th>
    </tr>

    <tr>
        <td>Name</td>
        <td>Mary</td>
    </tr>
</table>
```

Use `rowspan` to make a cell span multiple rows.

```
<table border="1">
    <tr>
        <th rowspan="2">ICT</th>
        <td>HTML</td>
    </tr>

    <tr>
        <td>Computer Basics</td>
    </tr>
</table>
```

---

11. Buttons in HTML

The `<button>` element creates a clickable button.

Basic Button

```
<button>Click Me</button>
```

By default, a button inside a form may submit the form. If it should perform another action, use `type="button"`.

Button with JavaScript

The following example changes a greeting when the user clicks the button.

```
<p id="greeting">Hello!</p>

<button type="button" onclick="doSomething()">
    Change Greeting
</button>

<script>
    function doSomething() {
        document.getElementById("greeting").innerHTML = "Goodbye!";
    }
</script>
```

Explanation

- `id="greeting"` gives the paragraph a unique identifier.
- `onclick="doSomething()"` calls a JavaScript function when the button is clicked.
- `function doSomething()` defines the function.
- `document.getElementById("greeting")` selects the paragraph.
- `.innerHTML = "Goodbye!"` changes its content.

Important: JavaScript function names are case-sensitive. Write `function`, not `Function`.

Making a Button-Like Link

If the purpose is to navigate to another page, use an anchor element:

```
<a href="about.html">Visit About Us</a>
```

An anchor can also contain a button, but this is not recommended because it nests two interactive elements. Use the anchor itself for navigation and a button for actions.

---

12. HTML Forms

Forms collect information from users. They are used for registration, login pages, surveys, feedback and many other purposes.

Basic Form Structure

```
<form action="process.php" method="post">
    <label for="username">Username:</label>
```

```
    <input type="text" id="username" name="username" required>

    <input type="submit" value="Submit">
</form>
```

Important Form Attributes

- `action` — Specifies the destination that processes the submitted data.
- `method` — Specifies how the data is sent, commonly `get` or `post`.
- `enctype` — Specifies the encoding used when submitting form data.
- `name` — Identifies a field in the submitted data.
- `id` — Uniquely identifies an element in the page.
- `required` — Requires the user to fill in a field before submission.
- `minlength` — Specifies the minimum number of characters for supported text fields.
- `placeholder` — Displays a hint inside an input field.

GET versus POST

- `GET` places submitted form values in the URL query string. It is suitable for searches and filters.
- `POST` sends form data in the request body. It is commonly used for registration and other data submissions.

A form needs a suitable server-side endpoint to process submitted data. HTML alone does not save registration details to a database.

12.1 Text Input

```
<label for="username">Username:</label>

<input type="text"
       id="username"
       name="username"
       minlength="6"
       required>
```

This field accepts ordinary text. The `minlength="6"` attribute requires at least six characters when the form is submitted through a supporting browser.

12.2 Password Input

```
<label for="password">Password:</label>

<input type="password"
       id="password"
       name="password"
```

```
       minlength="6"
       required>
```

The password input masks the characters displayed on screen. It does not encrypt the data by itself.

12.3 Email Input

```
<label for="email">Email Address:</label>

<input type="email"
       id="email"
       name="email"
       placeholder="example@email.com"
       required>
```

The browser checks whether the entered value resembles a valid email address.

12.4 Telephone Input with a Pattern

```
<label for="phone">Phone Number:</label>

<input type="tel"
       id="phone"
       name="phone"
       pattern="[0-9]{3}-[0-9]{3}-[0-9]{3}"
       placeholder="123-456-789"
       required>
```

The pattern requires three digits, a hyphen, three digits, another hyphen and three digits.

The pattern above is only an example format, not a complete Kenyan telephone-number validation rule.

12.5 Date Input

```
<label for="birthday">Date of Birth:</label>

<input type="date"
       id="birthday"
       name="birthday">
```

The browser provides a date-selection interface where supported.

12.6 Number Input

```
<label for="quantity">Quantity:</label>

<input type="number"
```

```
       id="quantity"
       name="quantity"
       min="0"
       max="99"
       required>
```

This accepts a number between 0 and 99.

12.7 Radio Buttons

Radio buttons allow the user to select one option from a group when the options share the same `name`.

```
<p>Title:</p>

<input type="radio" id="mr" name="title" value="Mr">
<label for="mr">Mr.</label>

<input type="radio" id="mrs" name="title" value="Mrs">
<label for="mrs">Mrs.</label>

<input type="radio" id="ms" name="title" value="Ms">
<label for="ms">Ms.</label>
```

The shared `name="title"` groups the radio buttons together.

12.8 Submit and Reset Buttons

```
<input type="submit" value="Submit Form">

<input type="reset" value="Clear Form">
```

- **Submit** sends the form data to the destination specified by the form's `action`.
- **Reset** restores the form fields to their initial values.

A reset button does not automatically delete previously saved data from a database.

12.9 Select Dropdown

A dropdown allows the user to choose from a list of options.

```
<label for="payment">Payment Method:</label>

<select id="payment" name="payment" required>
    <option value="">Choose a payment method</option>
    <option value="visa">Visa</option>
    <option value="mpesa">M-Pesa</option>
    <option value="cash">Cash</option>
</select>
```

Each `<option>` defines a choice. The `value` is the value submitted with the form.

12.10 Checkbox

A checkbox allows a user to select or deselect an option.

```
<input type="checkbox"
       id="subscribe"
       name="subscribe"
       value="yes">

<label for="subscribe">Subscribe to our newsletter</label>
```

Unlike radio buttons, checkboxes are suitable for independent choices. Multiple checkboxes can be selected.

12.11 Textarea

A `<textarea>` allows the user to enter longer text, such as comments or feedback.

```
<label for="comment">Comments:</label>

<textarea id="comment"
          name="comment"
          rows="4"
          cols="30"
          placeholder="Write your comments here"></textarea>
```

- `rows` specifies the approximate visible number of text lines.
- `cols` specifies the approximate visible width in characters.

Unlike `<input>`, a textarea has an opening and a closing tag.

12.12 File Upload

A file input allows the user to select a file from their device.

```
<label for="photo">Upload a Photo:</label>

<input type="file"
       id="photo"
       name="photo"
       accept="image/png, image/jpeg">
```

The `accept` attribute suggests which file types the user should select. It does not guarantee that an uploaded file is safe.

Uploading Files Through a Form

```
<form action="upload.php"
      method="post"
      enctype="multipart/form-data">

    <label for="photo">Choose an image:</label>

    <input type="file"
           id="photo"
           name="photo"
           accept="image/png, image/jpeg"
           required>

    <input type="submit" value="Upload">
</form>
```

The `enctype="multipart/form-data"` attribute is required for a standard HTML form to submit files correctly. A server-side script must handle and store the uploaded file.

---

13. Complete HTML Form Example

The following example combines several form elements into one student registration form.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Registration</title>
</head>

<body>

    <h1>Student Registration Form</h1>

    <form action="register.php"
          method="post"
          enctype="multipart/form-data">

        <label for="fullname">Full Name:</label>
        <input type="text"
               id="fullname"
               name="fullname"
               required>
        <br><br>

        <label for="email">Email:</label>
        <input type="email"
               id="email"
               name="email"
               required>
        <br><br>
```

```
<label for="password">Password:</label>
<input type="password"
       id="password"
       name="password"
       minlength="6"
       required>
<br><br>

<label for="phone">Phone Number:</label>
<input type="tel"
       id="phone"
       name="phone"
       required>
<br><br>

<label for="birthday">Date of Birth:</label>
<input type="date"
       id="birthday"
       name="birthday">
<br><br>

<label for="course">Choose Course:</label>

<select id="course" name="course" required>
    <option value="">Select a course</option>
    <option value="html">HTML</option>
    <option value="python">Python Programming</option>
    <option value="data-analysis">Data Analysis</option>
</select>
<br><br>

<p>Gender:</p>

<input type="radio"
       id="male"
       name="gender"
       value="Male">
<label for="male">Male</label>

<input type="radio"
       id="female"
       name="gender"
       value="Female">
<label for="female">Female</label>
<br><br>

<label for="comment">Additional Information:</label>
<br>

<textarea id="comment"
          name="comment"
          rows="4"
          cols="30"></textarea>
<br><br>

<label for="photo">Upload Photo:</label>
<input type="file"
```

```
                id="photo"
                name="photo"
                accept="image/png, image/jpeg">
        <br><br>

        <input type="checkbox"
                id="agree"
                name="agree"
                value="yes"
                required>

        <label for="agree">I confirm that the information is correct.</label>
        <br><br>

        <input type="submit" value="Register">
        <input type="reset" value="Clear Form">

    </form>

</body>
</html>
```

Important: This is the HTML structure of a registration form. The `register.php` file must be implemented on a server for the submitted information to be processed and saved. Do not collect real passwords without appropriate secure server-side handling.

---

14. General HTML Best Practices

1. Use lowercase tag and attribute names for consistency.
2. Put attribute values inside quotation marks.
3. Close elements that require closing tags.
4. Nest elements correctly.
5. Use meaningful `alt` text for informative images.
6. Use unique `id` values within a page.
7. Associate form labels with inputs using matching `for` and `id` values.
8. Give form controls a `name` attribute when their values need to be submitted.
9. Use `<a>` for navigation and `<button>` for actions.
10. Check all image, audio and video file paths carefully.
11. Use semantic elements such as `<strong>`, `<em>` and headings where appropriate.
12. Remember that HTML provides webpage structure; CSS controls presentation, while JavaScript adds interactivity.

CSS CLASS NOTES

Cascading Style Sheets

1. Introduction to CSS

CSS stands for **Cascading Style Sheets**. It is a stylesheet language used to control the appearance and layout of HTML webpages.

While HTML provides the structure and content of a webpage, CSS controls how that content looks.

For example:

- HTML creates headings, paragraphs, images, tables and forms.
- CSS changes colours, fonts, spacing, borders, sizes and positioning.

Example

HTML:

```
<h1>Welcome to RuralNet</h1>
<p>We offer computer training courses.</p>
```

CSS:

```
h1 {
    color: green;
}

p {
    font-size: 18px;
}
```

The heading will appear green, while the paragraph will have a font size of 18 pixels.

---

2. Basic CSS Syntax

A CSS rule consists of a selector and a declaration block.

Syntax

```
selector {
    property: value;
}
```

Example:

```
p {
```

```
    color: blue;
    font-size: 20px;
}
```

Explanation:

- `p` is the selector. It targets paragraph elements.
- `color` is a property that controls text colour.
- `blue` is the value assigned to the property.
- `font-size` is a property that controls text size.
- `20px` is its value.
- The curly brackets `{ }` contain the declarations.
- The colon `:` separates each property from its value.
- The semicolon `;` ends each declaration.

A declaration consists of a property and its value.

---

3. Methods of Adding CSS to HTML

There are three main methods of applying CSS to an HTML document.

3.1 Inline CSS

Inline CSS is written directly inside an HTML element using the `style` attribute.

Example:

```
<h1 style="color: red;">Welcome</h1>

<p style="font-size: 20px;">
    This is a paragraph.
</p>
```

Advantages

- Quick to apply to a single element.
- Useful for testing individual styles.

Disadvantages

- Difficult to maintain across many elements.
- Can make HTML code lengthy.
- Cannot easily apply one consistent rule to many elements.

3.2 Internal CSS

Internal CSS is written inside a `<style>` element, usually within the `<head>` section of an HTML document.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Internal CSS</title>

    <style>
        h1 {
            color: green;
        }

        p {
            font-size: 18px;
        }
    </style>
</head>
<body>

    <h1>Welcome to RuralNet</h1>
    <p>Learn computer skills with us.</p>

</body>
</html>
```

Internal CSS is suitable when styling a single webpage.

3.3 External CSS

External CSS is written in a separate file with the `.css` extension and linked to an HTML document.

Suppose the files are organized as follows:

```
MyWebsite/
│
├── index.html
└── style.css
```

In `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>External CSS</title>
    <link rel="stylesheet" href="style.css">
</head>
```

```
<body>

    <h1>Welcome to RuralNet</h1>
    <p>Learn computer skills with us.</p>

</body>
</html>
```

In `style.css`:

```
h1 {
    color: green;
}

p {
    font-size: 18px;
}
```

Advantages of external CSS

- One stylesheet can style multiple webpages.
- It keeps HTML code organized.
- It makes updating a website easier.
- It promotes consistency across pages.

Comparison of CSS Methods

Method	Where it is written	Best use
Inline	Inside an HTML element	One-off styling
Internal	Inside `<style>` in the HTML document	A single page
External	In a separate `.css` file	Multiple webpages

---

4. CSS Selectors

A selector identifies the HTML elements to which CSS rules should apply.

4.1 Element Selector

Targets all elements of a particular type.

```
p {
    color: blue;
}
```

This targets every paragraph on the page.

4.2 ID Selector

Targets an element with a particular `id`. An ID should be unique within a page.

In HTML:

```
<h1 id="heading">My Website</h1>
```

In CSS:

```
#heading {
    color: red;
}
```

The hash symbol `#` is used to select an ID.

4.3 Class Selector

Targets all elements with a particular class.

HTML:

```
<p class="important">Read this notice.</p>

<p class="important">Submit your work on time.</p>
```

CSS:

```
.important {
    color: red;
    font-weight: bold;
}
```

The dot `.` is used to select a class.

4.4 Universal Selector

Targets all elements.

```
* {
    font-family: Arial, sans-serif;
}
```

This sets the font family on all elements, although individual elements may override it.

4.5 Grouping Selector

Allows multiple selectors to share the same declarations.

```
h1, h2, p {
    font-family: Arial, sans-serif;
}
```

This applies the font family to headings and paragraphs.

4.6 Descendant Selector

Targets elements found inside another element.

```
div p {
    color: green;
}
```

This targets paragraphs located anywhere inside a `<div>`.

4.7 Child Selector

Targets direct children of a specified element.

```
div > p {
    color: blue;
}
```

This targets paragraphs that are direct children of a `<div>`, not paragraphs nested deeper inside other elements.

---

5. CSS Colours

The `color` property changes text colour, while `background-color` changes the background colour of an element.

Example

```
h1 {
    color: red;
}

body {
    background-color: lightyellow;
}

p {
    color: navy;
}
```

Common Ways to Specify Colours

Colour names

```
h1 {
    color: green;
}
```

Hexadecimal values

```
h1 {
    color: #008000;
}
```

RGB values

```
h1 {
    color: rgb(0, 128, 0);
}
```

RGBA values

```
div {
    background-color: rgba(0, 128, 0, 0.5);
}
```

RGBA includes an alpha value that controls opacity.

Examples of common colour values:

Colour	Hexadecimal
Black	`#000000`
White	`#FFFFFF`
Red	`#FF0000`
Green	`#008000`
Blue	`#0000FF`
Yellow	`#FFFF00`
Orange	`#FFA500`

---

6. Background Properties

CSS provides several properties for controlling an element's background.

6.1 Background Colour

```
body {
    background-color: lightblue;
```

```
}
```

6.2 Background Image

```
body {
    background-image: url("images/background.jpg");
}
```

6.3 Preventing Background Repetition

```
body {
    background-image: url("images/background.jpg");
    background-repeat: no-repeat;
}
```

6.4 Controlling Background Size

```
body {
    background-image: url("images/background.jpg");
    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
}
```

- `cover` scales the image to cover the container, possibly cropping some parts.
- `contain` scales the image to fit inside the container, possibly leaving empty space.
- `background-position` controls where the image is positioned.
- `background-repeat` controls whether the image repeats.

---

7. Text Properties

CSS can control the appearance and alignment of text.

7.1 Text Colour

```
p {
    color: darkgreen;
}
```

7.2 Text Alignment

```
h1 {
    text-align: center;
}
```

Common values are `left`, `right`, `center` and `justify`.

7.3 Text Decoration

```
h1 {
    text-decoration: underline;
}

a {
    text-decoration: none;
}
```

Other values include `overline` and `line-through`.

7.4 Text Transformation

```
h1 {
    text-transform: uppercase;
}
```

Common values:

- `uppercase` — converts text to uppercase visually.
- `lowercase` — converts text to lowercase visually.
- `capitalize` — capitalizes the first letter of each word.

These properties change the displayed text, not the original HTML content.

7.5 Text Indentation

```
p {
    text-indent: 30px;
}
```

This indents the first line of a paragraph.

7.6 Line Height

```
p {
    line-height: 1.6;
}
```

This controls the vertical space between lines of text.

---

8. Font Properties

8.1 Font Family

```
body {
    font-family: Arial, sans-serif;
}
```

Common font families include Arial, Verdana, Georgia, Times New Roman and Courier New.

8.2 Font Size

```
h1 {
    font-size: 36px;
}

p {
    font-size: 18px;
}
```

Common units include:

- `px` — pixels.
- `em` — relative to the applicable font size.
- `rem` — relative to the root element's font size.
- `%` — percentage-based sizing where supported by the property.

8.3 Font Weight

```
h1 {
    font-weight: bold;
}
```

Values include `normal`, `bold` and numeric weights such as `400` and `700`, where supported by the font.

8.4 Font Style

```
p {
    font-style: italic;
}
```

Common values are `normal`, `italic` and `oblique`.

Combined Example

```
h1 {
    font-family: Arial, sans-serif;
    font-size: 36px;
    font-weight: bold;
    font-style: italic;
    text-align: center;
    color: green;
}
```

---

9. CSS Units

CSS units specify measurements such as widths, heights, spacing and font sizes.

Unit	Meaning	Example
`px`	Pixels	`width: 200px;`
`%`	Percentage of a relevant containing dimension	`width: 80%;`
`em`	Relative to the applicable font size	`font-size: 1.5em;`
`rem`	Relative to the root font size	`font-size: 1.5rem;`
`vw`	Percentage of viewport width	`width: 50vw;`
`vh`	Percentage of viewport height	`height: 50vh;`

Example:

```
.container {
    width: 80%;
}

h1 {
    font-size: 2rem;
}

section {
    min-height: 50vh;
}
```

Relative units are often useful when designing responsive webpages.

---

10. Width and Height

The `width` and `height` properties control the dimensions of an element.

```
img {
    width: 300px;
    height: 200px;
}
```

To preserve an image's proportions, you can set one dimension and allow the other to adjust automatically.

```
img {
    width: 300px;
    height: auto;
```

```
}
```

Other useful properties include:

```
.box {
    min-width: 200px;
    max-width: 600px;
    min-height: 100px;
}
```

- `min-width` defines the minimum width.
- `max-width` defines the maximum width.
- `min-height` defines the minimum height.

---

11. The CSS Box Model

Every HTML element is represented as a rectangular box. The CSS box model describes its content, padding, border and margin.

The Four Parts

1. **Content:** The actual text, image or other content.
2. **Padding:** Space between the content and the border.
3. **Border:** The line surrounding the content and padding.
4. **Margin:** Space outside the border, separating the element from other elements.

Example

```
div {
    width: 300px;
    padding: 20px;
    border: 5px solid black;
    margin: 15px;
}
```

With the default `content-box` sizing model, the total width including padding and borders is:

300px + 40px padding + 10px border = 350px.

Margins add additional external space.

Using `box-sizing`

```
* {
    box-sizing: border-box;
}
```

With `border-box`, the declared width includes the content, padding and border. This makes layout calculations easier.

---

12. Borders

The `border` property adds a border around an element.

Example

```
p {
    border: 2px solid red;
}
```

The three parts are:

- `2px` — border thickness.
- `solid` — border style.
- `red` — border colour.

Common Border Styles

```
.box1 {
    border: 2px solid black;
}

.box2 {
    border: 2px dashed blue;
}

.box3 {
    border: 2px dotted green;
}

.box4 {
    border: 4px double red;
}
```

Border Radius

```
button {
    border-radius: 10px;
}
```

This rounds the corners of the button.

A circular shape can be created when the element's dimensions and radius are appropriate:

```
.circle {
    width: 100px;
    height: 100px;
    border-radius: 50%;
    background-color: green;
}
```

---

13. Margins and Padding

Margins and padding both create space, but they work differently.

Margin

Margin creates space outside an element's border.

```
h1 {
    margin: 20px;
}
```

Padding

Padding creates space inside the border, between the content and the border.

```
p {
    padding: 15px;
}
```

Individual Sides

```
div {
    margin-top: 10px;
    margin-right: 20px;
    margin-bottom: 10px;
    margin-left: 20px;
}
```

The same pattern works with `padding-top`, `padding-right`, `padding-bottom` and `padding-left`.

Shorthand Notation

```
div {
    margin: 10px 20px;
}
```

This means 10px top and bottom, and 20px left and right.

```
div {
```

```
    padding: 10px 15px 20px 25px;
}
```

The values represent top, right, bottom and left, respectively.

---

14. Styling Links

CSS can change the appearance of hyperlinks.

```
a {
    color: blue;
    text-decoration: none;
}

a:hover {
    color: red;
    text-decoration: underline;
}
```

Link States

- `a:link` — An unvisited link.
- `a:visited` — A visited link.
- `a:hover` — A link when the pointer is over it.
- `a:active` — A link while it is being activated.

Example:

```
a:link {
    color: blue;
}

a:visited {
    color: purple;
}

a:hover {
    color: red;
}

a:active {
    color: orange;
}
```

The `:hover` selector is a pseudo-class. It applies styles when the pointer is over the element.

---

15. Styling Buttons

Buttons can be styled using background colour, text colour, padding, borders and other properties.

HTML:

```
<button>Register Now</button>
```

CSS:

```
button {
    background-color: green;
    color: white;
    padding: 12px 24px;
    border: none;
    border-radius: 6px;
    font-size: 16px;
    cursor: pointer;
}

button:hover {
    background-color: darkgreen;
}
```

- `background-color` sets the button's background.
- `color` sets the text colour.
- `padding` creates internal space.
- `border-radius` rounds the corners.
- `cursor: pointer` indicates that the button is clickable.
- `:hover` changes the button's appearance when the pointer is over it.

---

16. Styling Lists

CSS can change list markers and spacing.

```
ul {
    list-style-type: square;
}

ol {
    list-style-type: upper-roman;
}
```

Common unordered-list values include `disc`, `circle` and `square`.

Common ordered-list values include `decimal`, `lower-alpha`, `upper-alpha`, `lower-roman` and `upper-roman`.

To remove list markers:

```
ul {
    list-style-type: none;
    padding-left: 0;
}
```

This technique is often used when creating navigation menus.

---

17. Styling Tables

CSS can improve the readability of HTML tables.

HTML:

```
<table>
    <tr>
        <th>Name</th>
        <th>Course</th>
    </tr>

    <tr>
        <td>Mary</td>
        <td>HTML</td>
    </tr>

    <tr>
        <td>John</td>
        <td>Python</td>
    </tr>
</table>
```

CSS:

```
table {
    width: 100%;
    border-collapse: collapse;
}

th, td {
    border: 1px solid black;
    padding: 10px;
    text-align: left;
}

th {
    background-color: green;
```

```
    color: white;
}

tr:hover {
    background-color: lightyellow;
}
```

- `width: 100%` makes the table fill its available width.
- `border-collapse: collapse` combines adjacent cell borders.
- `padding` creates space inside cells.
- `text-align` controls horizontal alignment.
- `tr:hover` highlights a row when the pointer is over it.

---

18. Display Property

The `display` property controls how an element participates in the layout.

Block Elements

Block elements normally start on a new line and take the available width.

```
div {
    display: block;
}
```

Examples include `<div>`, `<p>` and headings in their default styling.

Inline Elements

Inline elements normally remain within the same line and occupy the space their content needs.

```
span {
    display: inline;
}
```

Examples include `<span>` and `<a>` in their default styling.

Inline-Block

```
button {
    display: inline-block;
}
```

An inline-block element can sit alongside other inline content while accepting width and height settings.

Hiding an Element

```
.hidden {
    display: none;
}
```

The element is not displayed and does not occupy its normal layout space.

---

19. Position Property

The `position` property controls how an element is positioned.

Static

```
div {
    position: static;
}
```

This is the default positioning mode.

Relative

```
div {
    position: relative;
    left: 20px;
}
```

The element is shifted relative to its normal position while its original layout space remains reserved.

Absolute

```
.container {
    position: relative;
}

.box {
    position: absolute;
    top: 10px;
    right: 10px;
}
```

The absolutely positioned element is positioned relative to its nearest qualifying positioned ancestor, here `.container`.

Fixed

```
.chat-button {
    position: fixed;
    right: 20px;
    bottom: 20px;
}
```

The element is positioned relative to the viewport and normally remains in place while the page scrolls.

Sticky

```
header {
    position: sticky;
    top: 0;
}
```

The element behaves normally until it reaches the specified scrolling threshold, where it sticks within its scrolling context.

---

20. Flexbox

Flexbox is a CSS layout system used to arrange items in a row or column.

HTML:

```
<div class="container">
    <div>Home</div>
    <div>About</div>
    <div>Contact</div>
</div>
```

CSS:

```
.container {
    display: flex;
    justify-content: space-around;
    align-items: center;
    gap: 20px;
}
```

Important Flexbox properties:

- `display: flex` activates Flexbox for the container.
- `justify-content` controls distribution along the main axis.
- `align-items` controls alignment along the cross axis.
- `gap` sets space between items.
- `flex-direction` sets the main direction.

Example:

```
.container {
    display: flex;
    flex-direction: column;
    gap: 10px;
}
```

Common `flex-direction` values are `row`, `row-reverse`, `column` and `column-reverse`.

---

21. CSS Grid

CSS Grid is used to arrange content in rows and columns.

HTML:

```
<div class="grid-container">
    <div>Item 1</div>
    <div>Item 2</div>
    <div>Item 3</div>
    <div>Item 4</div>
</div>
```

CSS:

```
.grid-container {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 15px;
}
```

This creates two equal-width columns.

The `1fr` unit represents a fraction of the available space in the grid.

To create three equal columns:

```
.grid-container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 15px;
}
```

---

22. Responsive Web Design

Responsive design allows a webpage to adapt to different screen sizes, including phones, tablets and computers.

The Viewport Meta Tag

Add this element inside the HTML `<head>`:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

It helps browse;rs display pages at the device's actual viewport width.

Flexible Images

```
img {
    max-width: 100%;
    height: auto;
}
```

This prevents images from exceeding their containing element's width.

Media Queries

Media queries apply styles when specified conditions are met.

```
body {
    font-size: 18px;
}

@media (max-width: 600px) {
    body {
        font-size: 16px;
    }

    h1 {
        font-size: 24px;
    }
}
```

The styles inside the media query apply when the viewport is 600 pixels wide or narrower.

---

23. CSS Transitions

Transitions make changes in CSS properties appear smoother.

HTML:

```
<button class="my-button">Click Me</button>
```

CSS:

```
.my-button {
    background-color: blue;
    color: white;
    padding: 12px 20px;
    border: none;
    transition: background-color 0.3s ease;
}

.my-button:hover {
    background-color: darkblue;
}
```

The background colour changes smoothly over approximately 0.3 seconds when the pointer moves over the button.

---

24. CSS Comments

Comments help developers explain their code. Browsers ignore CSS comments.

Syntax:

```
/* This is a CSS comment */

h1 {
    color: green;
}

/* Style the main paragraph */
p {
    font-size: 18px;
}
```

CSS comments begin with `/*` and end with `*/`.

---

25. Common CSS Mistakes

1. Forgetting the opening or closing curly bracket.
2. Forgetting the colon between a property and its value.
3. Forgetting the semicolon between declarations.
4. Using `#` for a class selector instead of `.`.
5. Using `.` for an ID selector instead of `#`.
6. Linking the wrong CSS filename or path.
7. Misspelling a CSS property or value.

8. Forgetting quotation marks around a background image path when needed.
9. Confusing `margin` with `padding`.
10. Setting an image's width and height in a way that distorts it.

Incorrect CSS

```
p {
    color blue
    font-size: 20px
```

Correct CSS

```
p {
    color: blue;
    font-size: 20px;
}
```

---

26. Practical Exercise

Create a simple webpage for an ICT college using HTML and CSS.

Your webpage should contain:

1. A main heading displaying the college name.
2. A paragraph describing the college.
3. A logo or image.
4. A list of at least four courses.
5. A table containing three students and their courses.
6. Three links to Home, About Us and Contact Us.
7. A registration button.
8. A green navigation bar.
9. A coloured heading with a suitable font size.
10. A background colour of your choice.

Requirements

- Use external CSS in a file named `style.css`.
- Link the stylesheet correctly from your HTML document.
- Use at least one element selector, one class selector and one ID selector.
- Apply margins, padding, borders and text formatting.
- Make the image responsive.
- Ensure the page remains usable on a mobile screen.

Suggested Folder Structure

```
ICTCollege/
│
├── index.html
├── style.css
└── images/
    └── logo.png
```

= = = = =

MODERN CSS STYLING EFFECTS

1. GLASSMORPHISM

Definition: Glassmorphism is a CSS design technique that makes an element appear like transparent or frosted glass. It is commonly used on cards, navigation bars, login forms, and dashboards.

HTML

```
<div class="glass">
    <h2>Glassmorphism</h2>
    <p>This is a glass-style card</p>
</div>
```

CSS

```
body {
    background: linear-gradient(135deg, blue, purple);
    font-family: Arial, sans-serif;
    padding: 50px;
}

.glass {
    background: rgba(255, 255, 255, 0.2);
    backdrop-filter: blur(15px);
    -webkit-backdrop-filter: blur(15px);
    border: 1px solid rgba(255, 255, 255, 0.3);
    border-radius: 20px;
    padding: 25px;
    color: white;
}
```

Explanation of CSS Properties

CSS property	Explanation
`background`	Sets the background color or gradient behind the glass card.

`rgba()`	Defines a color using red, green, blue, and alpha (transparency).
`0.2` in `rgba()`	Makes the white background 20% opaque, allowing the background to show through.
`backdrop-filter: blur(15px)`	Blurs the content behind the element by 15 pixels.
`-webkit-backdrop-filter`	Provides compatibility with browsers that use the WebKit-prefixed version of this property.
`border`	Adds a thin border around the glass card.
`border-radius`	Rounds the corners of the card.
`padding`	Adds space between the card's content and its edges.
`color`	Sets the text color to white.

Expected result: A transparent, frosted-glass card with rounded corners and white text.

Note: Glassmorphism works best when a colorful background appears behind the element.

---

2. GLOW EFFECT

Definition: A glow effect makes an element appear as though it emits light. It can be applied to buttons, text, borders, and cards.

HTML

```
<h2 class="glow-text">WELCOME</h2>

<button class="glow-button">Click Me</button>
```

CSS

```
body {
    background: #0f172a;
    font-family: Arial, sans-serif;
}

.glow-text {
    color: cyan;
    text-shadow: 0 0 15px cyan;
}
```

```
.glow-button {
    background: blue;
    color: white;
    padding: 12px 25px;
    border: none;
    border-radius: 8px;
    box-shadow: 0 0 20px cyan;
}
```

Explanation of CSS Properties

CSS property	Explanation
`background`	Sets the page background to a dark navy color.
`color`	Sets the text color to cyan or white.
`text-shadow`	Adds a shadow or glowing effect around text.
`text-shadow: 0 0 15px cyan`	Creates a cyan glow without horizontal or vertical shadow displacement.
`padding`	Adds space inside the button.
`border: none`	Removes the button's default border.
`border-radius`	Rounds the corners of the button.
`box-shadow`	Adds a shadow around the button.
`box-shadow: 0 0 20px cyan`	Creates a cyan glow around the button.

Understanding `text-shadow`

Consider this declaration:

```
text-shadow: 3px 4px 5px black;
```

The values mean:

- `3px` — horizontal shadow displacement.
- `4px` — vertical shadow displacement.
- `5px` — blur radius.
- `black` — shadow color.

Expected result: The heading and button appear illuminated against the dark background.

---

3. BOX SHADOW

Definition: Box shadow adds a shadow around an HTML element, making it appear raised, elevated, or separated from the background.

HTML

```
<div class="card">
    <h2>Student Information</h2>
    <p>Name: John</p>
    <p>Course: ICT</p>
</div>
```

CSS

```
.card {
    background: white;
    width: 300px;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.2);
}
```

Explanation of CSS Properties

CSS property	Explanation
`background: white`	Gives the card a white background.
`width: 300px`	Sets the card width to 300 pixels.
`padding: 20px`	Adds 20 pixels of space inside the card.
`border-radius: 12px`	Rounds the card's corners.
`box-shadow`	Adds a shadow around the card.
First `0`	Horizontal shadow displacement.
Second `10px`	Vertical shadow displacement.
`25px`	Shadow blur radius.
`rgba(0, 0, 0, 0.2)`	Sets the shadow color to black with 20% opacity.

Types of Box Shadow

A. Soft shadow

```
box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
```

Creates a light, subtle shadow.

B. Deep shadow

```
box-shadow: 0 15px 30px rgba(0, 0, 0, 0.3);
```

Creates a darker, more pronounced shadow.

C. Inset shadow

```
box-shadow: inset 0 3px 8px rgba(0, 0, 0, 0.3);
```

Creates a shadow inside the element, making it appear pressed inward.

D. Colored shadow

```
box-shadow: 0 5px 20px rgba(0, 0, 255, 0.5);
```

Creates a blue shadow.

Expected result: The card appears elevated above the page background.

---

4. GRADIENT BACKGROUND

Definition: A gradient is a gradual transition between two or more colors. CSS supports linear gradients, radial gradients, and conic gradients.

HTML

```
<div class="gradient-box">
    <h2>Modern Design</h2>
    <p>Beautiful gradient background.</p>
</div>
```

CSS

```
.gradient-box {
    background: linear-gradient(135deg, blue, cyan);
    color: white;
```

```
    padding: 30px;
    border-radius: 15px;
}
```

Explanation of CSS Properties

CSS property	Explanation
`background`	Sets the background of the element.
`linear-gradient()`	Blends colors along a straight line.
`135deg`	Sets the direction of the gradient to 135 degrees.
`blue, cyan`	Specifies the two colors being blended.
`color: white`	Makes the text white.
`padding`	Adds space inside the box.
`border-radius`	Rounds the corners.

Other Gradient Styles

A. Horizontal gradient

```
background: linear-gradient(to right, red, yellow);
```

Blends red into yellow from left to right.

B. Vertical gradient

```
background: linear-gradient(to bottom, green, white);
```

Blends green into white from top to bottom.

C. Radial gradient

```
background: radial-gradient(circle, yellow, red);
```

Creates a circular color transition from yellow to red.

D. Three-color gradient

```
background: linear-gradient(90deg, blue, purple, pink);
```

Blends three colors from left to right.

Expected result: The element displays a smooth transition between the specified colors.

---

5. GRADIENT TEXT

Definition: Gradient text uses two or more colors to fill the letters instead of displaying a single text color.

HTML

```
<h1 class="gradient-text">RURALNET ICT ACADEMY</h1>
```

CSS

```
.gradient-text {
    background: linear-gradient(to right, blue, cyan, purple);
    background-clip: text;
    -webkit-background-clip: text;
    color: transparent;
}
```

Explanation of CSS Properties

CSS property	Explanation
`background`	Defines the gradient used to color the text.
`linear-gradient()`	Creates a gradual transition between colors.
`background-clip: text`	Clips the background so that it appears within the text characters.
`-webkit-background-clip: text`	Supports the text-clipping effect in WebKit-based browsers.
`color: transparent`	Makes the original text color transparent, allowing the gradient to show through.

Expected result: The heading displays blue, cyan, and purple colors across its letters.

---

6. NEUMORPHISM

Definition: Neumorphism is a design style that makes elements appear raised from or pressed into their background using light and dark shadows.

HTML

```
<div class="neu-card">
    <h2>Neumorphism</h2>
    <button class="neu-button">Submit</button>
</div>
```

CSS

```
body {
    background: #e0e5ec;
}

.neu-card {
    background: #e0e5ec;
    padding: 30px;
    border-radius: 20px;

    box-shadow:
        8px 8px 16px #bec3c9,
        -8px -8px 16px #ffffff;
}

.neu-button {
    background: #e0e5ec;
    padding: 12px 20px;
    border: none;
    border-radius: 10px;

    box-shadow:
        4px 4px 8px #bec3c9,
        -4px -4px 8px #ffffff;
}
```

Explanation of CSS Properties

CSS property	Explanation
`background: #e0e5ec`	Gives the card and page the same light gray background.
`padding`	Creates space inside the elements.
`border-radius`	Rounds the corners.
`box-shadow`	Creates the raised appearance using two shadows.
First shadow	A darker shadow that makes the element appear raised.

CSS property	Explanation
Second shadow	A lighter shadow that simulates light shining on the opposite side.
`border: none`	Removes the button's default border.

Expected result: The card and button appear raised from the gray background.

Note: Neumorphism is usually most effective when the element and its background have similar colors.

---

7. HOVER EFFECT

Definition: A hover effect changes the appearance of an element when the mouse pointer moves over it.

HTML

```
<button class="hover-button">Hover Over Me</button>
```

CSS

```
.hover-button {
    background: blue;
    color: white;
    padding: 15px 25px;
    border: none;
    border-radius: 8px;

    transition: all 0.3s ease;
}

.hover-button:hover {
    background: cyan;
    color: black;
    transform: translateY(-5px);
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.3);
}
```

Explanation of CSS Properties

CSS property	Explanation
`transition`	Makes changes occur smoothly rather than instantly.

`0.3s`	Sets the transition duration to 0.3 seconds.
`ease`	Controls the transition speed, making it start and finish more gently.
`:hover`	Applies styles when the mouse pointer is over the element.
`background: cyan`	Changes the button background to cyan.
`color: black`	Changes the text color to black.
`transform`	Changes the element's position, size, rotation, or shape.
`translateY(-5px)`	Moves the button upward by 5 pixels.
`box-shadow`	Adds a shadow when the button is hovered over.

Expected result: When the user moves the mouse over the button, it changes color, moves upward, and displays a shadow.

---

8. CSS BLUR EFFECT

Definition: Blur is a visual effect that makes an element or the background behind it appear less sharp.

There are two commonly used blur properties.

A. Filter Blur

HTML

```
<img src="student.jpg" class="blur-image" alt="Student">
```

CSS

```
.blur-image {
    filter: blur(5px);
}
```

Explanation

CSS property	Explanation
`filter`	Applies a visual effect to the element itself.
`blur(5px)`	Blurs the image with a blur radius of 5 pixels.

Result: The image becomes blurred.

B. Background Blur

HTML

```
<div class="blur-card">
    <h2>Login Form</h2>
    <p>Enter your login details.</p>
</div>
```

CSS

```
.blur-card {
    background: rgba(255, 255, 255, 0.2);
    backdrop-filter: blur(15px);
    -webkit-backdrop-filter: blur(15px);

    padding: 25px;
    border-radius: 15px;
}
```

Explanation

CSS property	Explanation
`background`	Gives the card a transparent white background.
`backdrop-filter`	Applies a visual effect to the content behind the element.
`blur(15px)`	Blurs the background behind the card.
`padding`	Adds internal spacing.
`border-radius`	Rounds the corners.

Result: The card appears like frosted glass while the content behind it is blurred.

Important difference: `filter: blur()` blurs the element itself, while `backdrop-filter: blur()` blurs the area behind it.

9. BORDER EFFECTS

Definition: CSS borders define the edges of an HTML element. They can be customized using color, width, style, and rounded corners.

HTML

```
<div class="border-card">
    <h2>Border Styling</h2>
    <p>This card has a colored border.</p>
</div>
```

CSS

```
.border-card {
    background: white;
    padding: 25px;

    border: 3px solid blue;
    border-radius: 15px;
}
```

Explanation of CSS Properties

CSS property	Explanation
`border`	Defines the border around an element.
`3px`	Sets the border thickness to 3 pixels.
`solid`	Specifies a continuous border line.
`blue`	Sets the border color to blue.
`border-radius`	Rounds the corners of the element.

Other Border Styles

A. Dotted border

```
border: 2px dotted red;
```

B. Dashed border

```
border: 2px dashed green;
```

C. Double border

```
border: 4px double purple;
```

D. No border

```
border: none;
```

E. Rounded border

```
border: 2px solid blue;
border-radius: 20px;
```

10. TRANSFORM EFFECTS

Definition: The CSS `transform` property changes the position, size, rotation, or shape of an element without changing the normal document flow.

HTML

```
<div class="transform-box">
    <h2>Transform Effects</h2>
</div>
```

CSS

```
.transform-box {
    background: blue;
    color: white;
    width: 250px;
    padding: 20px;
    transition: transform 0.3s ease;
}

.transform-box:hover {
    transform: scale(1.05);
}
```

Explanation of CSS Properties

CSS property	Explanation
`transition`	Makes the transformation smooth.
`transform`	Applies a visual transformation to the element.
`scale(1.05)`	Enlarges the element to 105% of its original size.
`:hover`	Applies the transformation when the pointer is over the element.

Other Transform Values

CSS code	Function
`transform: scale(1.2);`	Enlarges an element by 20%.
`transform: scale(0.8);`	Reduces the element to 80% of its original size.

`transform: rotate(45deg);`	Rotates the element 45 degrees clockwise.
`transform: translateX(20px);`	Moves the element 20 pixels to the right.
`transform: translateY(-10px);`	Moves the element 10 pixels upward.
`transform: skew(10deg);`	Skews the element horizontally.

11. OPACITY AND TRANSPARENCY

Definition: Opacity determines how transparent or visible an element appears.

HTML

```
<div class="transparent-box">
    <h2>Transparency</h2>
    <p>This element has reduced opacity.</p>
</div>
```

CSS

```
.transparent-box {
    background: blue;
    color: white;
    padding: 25px;
    opacity: 0.5;
}
```

Explanation of CSS Properties

CSS property	Explanation
`opacity`	Controls the transparency of the entire element, including its contents.
`opacity: 1`	Makes the element fully opaque.
`opacity: 0.5`	Makes the element 50% opaque.
`opacity: 0`	Makes the element completely transparent.

Opacity Values

Value	Result
`opacity: 1;`	Fully visible
`opacity: 0.8;`	Slightly transparent
`opacity: 0.5;`	Half transparent
`opacity: 0.2;`	Highly transparent
`opacity: 0;`	Invisible

Important: Unlike `background: rgba(...)`, the `opacity` property affects the entire element, including its text and child elements.

---

12. CSS ANIMATION

Definition: Animation allows an HTML element to change its style automatically over a specified period.

HTML

```
<div class="animated-box">
    <h2>CSS Animation</h2>
</div>
```

CSS

```
.animated-box {
    background: blue;
    color: white;
    padding: 25px;
    border-radius: 15px;

    animation: moveBox 3s infinite alternate;
}

@keyframes moveBox {
    from {
        transform: translateX(0);
    }

    to {
        transform: translateX(100px);
    }
}
```

Explanation of CSS Properties

CSS property	Explanation
`animation`	Applies an animation to the element.
`moveBox`	The name given to the animation.
`3s`	Specifies that one animation cycle takes 3 seconds.
`infinite`	Repeats the animation continuously.
`alternate`	Makes the animation reverse direction on alternate cycles.
`@keyframes`	Defines the stages of the animation.
`from`	Defines the starting state of the animation.
`to`	Defines the ending state of the animation.
`translateX(100px)`	Moves the element 100 pixels to the right.

13. COMBINING MULTIPLE CSS EFFECTS

You can combine glassmorphism, shadows, gradients, and hover effects to create a modern card.

HTML

```
<div class="modern-card">
    <h2>Student Portal</h2>
    <p>Welcome to your dashboard.</p>
    <button class="modern-button">Get Started</button>
</div>
```

CSS

```
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #0f172a, #2563eb);
    padding: 50px;
}

.modern-card {
    background: rgba(255, 255, 255, 0.15);
```

```
    backdrop-filter: blur(15px);
    -webkit-backdrop-filter: blur(15px);

    border: 1px solid rgba(255, 255, 255, 0.25);
    border-radius: 20px;

    padding: 30px;
    color: white;

    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);

    transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.modern-card:hover {
    transform: translateY(-5px);

    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.3);
}

.modern-button {
    background: linear-gradient(90deg, blue, cyan);
    color: white;

    padding: 12px 25px;
    border: none;
    border-radius: 8px;

    cursor: pointer;
    transition: 0.3s ease;
}

.modern-button:hover {
    box-shadow: 0 0 20px cyan;
    transform: scale(1.05);
}
```

Summary of the Combined Properties

Property	Application in the example
`linear-gradient()`	Creates a colorful page background and button.
`rgba()`	Makes the card background transparent.
`backdrop-filter`	Creates the frosted-glass effect.
`border`	Adds a subtle border around the card.
`border-radius`	Rounds the card and button corners.
`box-shadow`	Creates depth around the card.

`transition`	Smooths the hover changes.
`transform`	Moves the card and enlarges the button.
`cursor: pointer`	Makes the button display a pointer cursor.

14. SUMMARY OF IMPORTANT CSS PROPERTIES

Property	Purpose	Example
`background`	Sets the background	`background: blue;`
`background-color`	Sets a solid background color	`background-color: white;`
`background-image`	Sets an image or gradient background	`background-image: linear-gradient(blue, cyan);`
`backdrop-filter`	Applies effects behind an element	`backdrop-filter: blur(10px);`
`box-shadow`	Adds a shadow or glow around a box	`box-shadow: 0 5px 15px gray;`
`text-shadow`	Adds a shadow or glow to text	`text-shadow: 2px 2px 5px black;`
`filter`	Applies visual effects to an element	`filter: blur(3px);`
`border`	Defines an element's border	`border: 2px solid red;`
`border-radius`	Rounds corners	`border-radius: 15px;`
`opacity`	Controls element transparency	`opacity: 0.5;`
`transform`	Moves, rotates, or scales an element	`transform: scale(1.1);`
`transition`	Smooths style changes	`transition: 0.3s ease;`
`animation`	Animates an element	`animation: moveBox 2s infinite;`

`cursor`	Changes the mouse cursor	`cursor: pointer;`
`padding`	Adds space inside an element	`padding: 20px;`
`margin`	Adds space outside an element	`margin: 20px;`

CSS NOTES – PART TWO: INTERMEDIATE AND ADVANCED CSS

1. CSS CASCADE, SPECIFICITY AND INHERITANCE

CSS uses rules to determine which styles should apply when several rules target the same HTML element.

A. The CSS Cascade

The cascade is the process through which the browser decides which CSS declaration takes priority when multiple declarations conflict.

Example:

```
p {
    color: blue;
}

p {
    color: red;
}
```

The paragraph will appear red because both selectors have equal specificity, and the second rule comes later.

B. CSS Specificity

Specificity determines which selector has more influence when different CSS rules target the same element.

General specificity order:

1. Inline styles – highest priority among ordinary author styles.
2. ID selectors.
3. Class, attribute and pseudo-class selectors.

4. Element and pseudo-element selectors.
5. Universal selector (`*`) – lowest specificity.

Example:

```
p {
    color: blue;
}

.text {
    color: green;
}

#message {
    color: red;
}
```

HTML:

```
<p id="message" class="text">Welcome to CSS</p>
```

The text will be red because the ID selector has higher specificity.

Note: `!important` can override ordinary declarations, but it should be used sparingly because it makes styles harder to maintain.

```
p {
    color: red !important;
}
```

C. CSS Inheritance

Inheritance occurs when some CSS properties applied to a parent element are automatically passed down to its children.

Example:

```
body {
    color: blue;
    font-family: Arial;
}
```

Text inside paragraphs, headings and other child elements will generally inherit the blue colour and Arial font unless another rule overrides them.

Commonly inherited properties include:

- `color`
- `font-family`
- `font-size`

- `line-height`
- `text-align`

Properties such as `margin`, `padding` and `border` are not normally inherited.

---

2. CSS PSEUDO-CLASSES

A pseudo-class is used to style an element when it is in a particular state or when it meets a certain condition.

Pseudo-classes begin with a single colon (`:`).

Common pseudo-classes

Pseudo-class	Purpose
`:hover`	When the mouse pointer is over an element
`:active`	When an element is being activated or clicked
`:focus`	When an element receives keyboard or input focus
`:visited`	A link that has already been visited
`:first-child`	Selects an element that is the first child
`:last-child`	Selects an element that is the last child
`:nth-child()`	Selects an element based on its position
`:checked`	Selects a checked checkbox or radio button
`:disabled`	Selects a disabled form element
`:not()`	Excludes elements matching a selector

Example 1: Hover effect

```
button {
    background-color: blue;
    color: white;
}

button:hover {
    background-color: darkblue;
}
```

The button changes colour when the mouse pointer moves over it.

Example 2: Styling links

```
a:link {
    color: blue;
```

```
}

a:visited {
    color: purple;
}

a:hover {
    color: red;
}

a:active {
    color: green;
}
```

Example 3: Selecting specific children

```
li:first-child {
    color: red;
}

li:last-child {
    color: blue;
}

li:nth-child(2) {
    color: green;
}
```

Example 4: Form focus

```
input:focus {
    border: 2px solid blue;
    outline: none;
}
```

The input border changes when the user clicks inside the input field or navigates to it using the keyboard.

---

3. CSS PSEUDO-ELEMENTS

A pseudo-element is used to style a specific part of an element or insert decorative content without adding another HTML element.

Pseudo-elements generally use two colons (`::`).

Common pseudo-elements

Pseudo-element	Purpose

`::before`	Inserts content before an element's content
`::after`	Inserts content after an element's content
`::first-letter`	Styles the first letter
`::first-line`	Styles the first line
`::selection`	Styles text selected by the user
`::placeholder`	Styles placeholder text in form fields

Example 1: First letter

```
p::first-letter {
    font-size: 30px;
    color: red;
    font-weight: bold;
}
```

Example 2: Before and after

```
h2::before {
    content: "★ ";
    color: gold;
}

h2::after {
    content: " ★";
    color: gold;
}
```

HTML:

```
<h2>Welcome to RuralNet</h2>
```

The symbols are added through CSS rather than being typed into the HTML.

Example 3: Selection styling

```
::selection {
    background-color: yellow;
    color: black;
}
```

This changes the appearance of text selected by the user.

Important: The `content` property is commonly used with `::before` and `::after`. These pseudo-elements are best used for decorative content rather than important information.

---

4. CSS ATTRIBUTE SELECTORS

Attribute selectors allow you to style HTML elements based on their attributes or attribute values.

Common attribute selector patterns

Selector	Meaning
`[type]`	Elements that have a type attribute
`[type="text"]`	Elements whose type is exactly text
`[href^="https"]`	Links whose href begins with https
`[href$=".pdf"]`	Links whose href ends with .pdf
`[class*="box"]`	Elements whose class contains box

Example 1: Selecting text inputs

```
input[type="text"] {
    border: 2px solid blue;
}
```

Example 2: Selecting email inputs

```
input[type="email"] {
    background-color: lightyellow;
}
```

Example 3: Selecting PDF links

```
a[href$=".pdf"] {
    color: red;
    font-weight: bold;
}
```

Attribute selectors are useful when styling form elements and links without assigning additional classes.

---

5. CSS GRADIENTS

A gradient is a gradual transition between two or more colours.

CSS gradients are commonly used as backgrounds.

A. Linear Gradient

A linear gradient blends colours along a straight line.

Syntax:

```
background: linear-gradient(direction, color1, color2);
```

Example:

```
body {
    background: linear-gradient(to right, blue, purple);
}
```

B. Changing the Direction

```
.box1 {
    background: linear-gradient(to right, red, yellow);
}

.box2 {
    background: linear-gradient(to bottom, blue, white);
}

.box3 {
    background: linear-gradient(45deg, green, yellow);
}
```

C. Multiple Colours

```
header {
    background: linear-gradient(
        to right,
        red,
        yellow,
        green,
        blue
    );
}
```

D. Radial Gradient

A radial gradient spreads out from a central point.

```
.box {
    background: radial-gradient(circle, yellow, red);
}
```

E. Conic Gradient

A conic gradient rotates around a central point.

```
.box {
```

```
    background: conic-gradient(
        red,
        yellow,
        green,
        blue,
        red
    );
}
```

Application: Gradients are useful when creating website banners, buttons, backgrounds and modern interface designs.

---

6. CSS BOX SHADOW AND TEXT SHADOW

A. Box Shadow

The `box-shadow` property adds a shadow around an element's box.

Syntax:

```
box-shadow: horizontal vertical blur spread color;
```

Example:

```
.card {
    width: 250px;
    padding: 20px;
    background-color: white;
    box-shadow: 5px 5px 10px gray;
}
```

Explanation:

- First value: horizontal shadow position.
- Second value: vertical shadow position.
- Third value: blur amount.
- Fourth value: spread amount.
- Fifth value: shadow colour.

Example with a softer shadow:

```
.card {
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.2);
}
```

B. Text Shadow

The `text-shadow` property adds a shadow to text.

Syntax:

```
text-shadow: horizontal vertical blur color;
```

Example:

```
h1 {
    color: blue;
    text-shadow: 2px 2px 4px gray;
}
```

C. Multiple Shadows

```
h1 {
    text-shadow: 2px 2px 3px gray,
                 4px 4px 5px lightblue;
}
```

---

7. CSS OPACITY AND VISIBILITY

These properties control whether an element is visible and how transparent it appears.

A. Opacity

The `opacity` property controls the transparency of an element.

Values range from `0` to `1`.

- `0` – Completely transparent.
- `0.5` – 50% opacity.
- `1` – Fully visible.

Example:

```
img {
    opacity: 0.5;
}
```

B. Visibility

The `visibility` property determines whether an element is visible.

```
.hidden {
    visibility: hidden;
}
```

The element becomes invisible but still occupies space in the layout.

C. Display None

```
.hidden {
    display: none;
}
```

The element disappears and no longer occupies space in the layout.

Difference

Property	Visible?	Occupies layout space?
`opacity: 0`	No, fully transparent	Yes
`visibility: hidden`	No	Yes
`display: none`	No	No

---

8. CSS OVERFLOW

The `overflow` property controls what happens when content is larger than the space allocated to an element.

Common overflow values

Value	Purpose
`visible`	Content can extend outside the element
`hidden`	Extra content is clipped
`scroll`	Scrollbars are provided
`auto`	Scrollbars appear when necessary

Example:

```
.box {
    width: 200px;
    height: 100px;
    overflow: auto;
}
```

Horizontal and Vertical Overflow

You can control the two directions separately.

```
.box {
    overflow-x: auto;
```

```
    overflow-y: hidden;
}
```

Application: Overflow is useful for scrollable sections, long tables, image galleries and content containers.

---

9. CSS Z-INDEX

The `z-index` property controls the stacking order of overlapping elements.

An element with a higher stacking level generally appears in front of an element with a lower stacking level, provided they participate in the relevant stacking context.

Example:

```
.box1 {
    position: absolute;
    z-index: 1;
}

.box2 {
    position: absolute;
    z-index: 5;
}
```

Box 2 will generally appear above Box 1 if they overlap in the same stacking context.

Practical Example

```
<div class="box1">First Box</div>
<div class="box2">Second Box</div>
.box1 {
    position: absolute;
    background-color: red;
    width: 150px;
    height: 150px;
    z-index: 1;
}

.box2 {
    position: absolute;
    background-color: blue;
    width: 150px;
    height: 150px;
    left: 50px;
    top: 50px;
    z-index: 2;
}
```

The blue box appears above the red box where they overlap.

Note: `z-index` does not work as expected in every situation. Stacking contexts created by properties such as positioned elements with a z-index, transforms and opacity can affect which element appears in front.

---

10. CSS TRANSFORM

The `transform` property changes the appearance or position of an element without changing the normal document flow.

Common transform functions

Function	Purpose
`translate()`	Moves an element
`rotate()`	Rotates an element
`scale()`	Increases or decreases its size
`skew()`	Slants an element

A. Translate

```
.box {
    transform: translate(50px, 20px);
}
```

Moves the element 50px horizontally and 20px vertically.

B. Rotate

```
.box {
    transform: rotate(45deg);
}
```

Rotates the element by 45 degrees.

C. Scale

```
.box {
    transform: scale(1.5);
}
```

Increases the element's size by 1.5 times.

D. Skew

```
.box {
    transform: skew(20deg);
}
```

Slants the element.

E. Combining Transforms

```
.box {
    transform: translate(20px, 10px) rotate(15deg) scale(1.2);
}
```

---

11. CSS ANIMATIONS

CSS animations allow elements to change their appearance or position over time without requiring JavaScript.

Animations are created using the `@keyframes` rule.

Basic Syntax

```
@keyframes animationName {
    from {
        /* Starting style */
    }

    to {
        /* Ending style */
    }
}
```

Example 1: Changing Colours

```
@keyframes changeColor {
    from {
        background-color: red;
    }

    to {
        background-color: blue;
    }
}

.box {
    width: 150px;
    height: 150px;
    animation: changeColor 3s;
}
```

Example 2: Moving an Element

```
@keyframes moveBox {
    from {
        transform: translateX(0);
    }

    to {
        transform: translateX(200px);
    }
}

.box {
    animation: moveBox 2s;
}
```

Animation Properties

Property	Purpose
`animation-name`	Specifies the animation
`animation-duration`	Sets how long it takes
`animation-delay`	Sets when it starts
`animation-iteration-count`	Sets how many times it runs
`animation-direction`	Controls the direction
`animation-timing-function`	Controls the speed pattern
`animation-fill-mode`	Controls styles before or after the animation

Example 3: Infinite Animation

```
@keyframes bounce {
    0% {
        transform: translateY(0);
    }

    50% {
        transform: translateY(-30px);
    }

    100% {
        transform: translateY(0);
    }
}

.ball {
    animation: bounce 1s infinite;
}
```

The ball moves up and down continuously.

---

12. CSS VARIABLES (CUSTOM PROPERTIES)

CSS variables allow you to store values and reuse them throughout a stylesheet.

They are declared using two hyphens (`--`) and accessed using `var()`.

Example

```
:root {
    --main-color: green;
    --secondary-color: orange;
    --main-font: Arial;
}

h1 {
    color: var(--main-color);
}

button {
    background-color: var(--secondary-color);
    font-family: var(--main-font);
}
```

Benefits of CSS Variables

1. Reduce repetition.
2. Make websites easier to maintain.
3. Allow colours and other values to be changed from one location.
4. Help maintain consistent website designs.

Changing a Variable

```
:root {
    --main-color: blue;
}

body {
    background-color: var(--main-color);
}
```

To change the background colour, you only need to update the variable.

---

13. CSS FUNCTIONS

CSS functions perform calculations or determine values dynamically.

A. calc()

The `calc()` function performs mathematical calculations.

```
.container {
    width: calc(100% - 40px);
}
```

The browser calculates the width by subtracting 40px from the available 100%.

B. min()

The `min()` function selects the smallest value.

```
.box {
    width: min(90%, 600px);
}
```

The box will use whichever value is smaller: 90% of its available space or 600px.

C. max()

The `max()` function selects the largest value.

```
.box {
    width: max(50%, 300px);
}
```

D. clamp()

The `clamp()` function sets a minimum, preferred and maximum value.

Syntax:

```
clamp(minimum, preferred, maximum);
```

Example:

```
h1 {
    font-size: clamp(24px, 5vw, 50px);
}
```

The heading will not become smaller than 24px or larger than 50px. Between those limits, its size responds to the viewport width.

---

14. ADVANCED FLEXBOX

Flexbox is a CSS layout system used to arrange items in rows or columns.

A. flex-wrap

Controls whether items should move to another line when there is insufficient space.

```
.container {
    display: flex;
    flex-wrap: wrap;
}
```

Values:

- nowrap
- wrap
- wrap-reverse

B. flex-direction

Controls the direction of the items.

```
.container {
    display: flex;
    flex-direction: column;
}
```

Values:

- row
- row-reverse
- column
- column-reverse

C. gap

Creates space between flex items.

```
.container {
    display: flex;
    gap: 20px;
}
```

D. flex-grow

Determines how much an item can grow to occupy available space.

```
.item {
    flex-grow: 1;
}
```

E. flex-shrink

Determines how much an item can shrink when space is limited.

```
.item {
    flex-shrink: 0;
}
```

F. flex-basis

Sets the initial size of a flex item before remaining space is distributed.

```
.item {
    flex-basis: 200px;
}
```

G. align-self

Allows an individual flex item to have a different alignment from the other items.

```
.item {
    align-self: flex-end;
}
```

Practical Example

```
<div class="container">
    <div class="item">Home</div>
    <div class="item">About</div>
    <div class="item">Contact</div>
</div>
.container {
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: 15px;
}

.item {
    background-color: green;
    color: white;
    padding: 20px;
}
```

---

15. ADVANCED CSS GRID

CSS Grid is a two-dimensional layout system that arranges items into rows and columns.

A. Defining Grid Columns

```
.container {
    display: grid;
    grid-template-columns: 200px 200px 200px;
}
```

This creates three columns, each 200px wide.

B. Using Fractional Units

The `fr` unit represents a fraction of the available space.

```
.container {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
}
```

This creates three equal columns.

C. Repeat Function

```
.container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
}
```

This is a shorter way of creating three equal columns.

D. Grid Gap

```
.container {
    display: grid;
    gap: 20px;
}
```

E. Grid Rows

```
.container {
    display: grid;
    grid-template-rows: 100px 200px;
}
```

F. Spanning Columns

```
.item1 {
    grid-column: span 2;
}
```

The item occupies two columns.

G. Responsive Grid

```
.container {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 20px;
}
```

This allows the browser to fit as many columns as possible, with each column having a minimum width of 200px.

---

16. STYLING HTML FORMS

CSS can be used to make forms attractive, readable and easy to use.

Example

HTML:

```
<form>
    <label for="name">Full Name:</label>
    <input type="text" id="name" placeholder="Enter your name">

    <label for="email">Email:</label>
    <input type="email" id="email" placeholder="Enter your email">

    <button type="submit">Register</button>
</form>
```

CSS:

```
form {
    width: 350px;
    padding: 20px;
    background-color: #f2f2f2;
    border-radius: 10px;
}

label {
    display: block;
    margin-top: 15px;
}

input {
    width: 100%;
    padding: 10px;
    margin-top: 5px;
    box-sizing: border-box;
    border: 1px solid gray;
    border-radius: 5px;
}
```

```
input:focus {
    border-color: blue;
    outline: none;
}

button {
    width: 100%;
    padding: 12px;
    margin-top: 20px;
    background-color: green;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}
```

Styling Different Input Types

```
input[type="checkbox"] {
    width: auto;
}

input[type="radio"] {
    accent-color: green;
}

input[type="file"] {
    border: none;
}
```

Important: CSS controls the appearance of forms. HTML handles the form structure, while server-side programming handles the processing and storage of submitted information.

---

17. CSS NAVIGATION MENUS

Navigation menus help users move between different pages or sections of a website.

Example: Horizontal Navigation Bar

HTML:

```
<nav>
    <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="courses.html">Courses</a></li>
        <li><a href="contact.html">Contact</a></li>
    </ul>
</nav>
```

CSS:

```
nav {
    background-color: darkgreen;
}

nav ul {
    list-style: none;
    display: flex;
    gap: 20px;
    margin: 0;
    padding: 15px;
}

nav a {
    color: white;
    text-decoration: none;
}

nav a:hover {
    color: yellow;
}
```

Vertical Navigation Menu

```
nav ul {
    display: flex;
    flex-direction: column;
    list-style: none;
}
```

Dropdown Menu

HTML:

```
<nav>
    <ul>
        <li><a href="#">Home</a></li>

        <li class="dropdown">
            <a href="#">Courses</a>

            <ul class="dropdown-menu">
                <li><a href="#">Web Development</a></li>
                <li><a href="#">Data Analysis</a></li>
                <li><a href="#">Graphic Design</a></li>
            </ul>
        </li>
    </ul>
</nav>
```

CSS:

```
.dropdown {
    position: relative;
}

.dropdown-menu {
    display: none;
    position: absolute;
    background-color: darkgreen;
    min-width: 180px;
    padding: 10px;
}

.dropdown:hover .dropdown-menu {
    display: block;
}

.dropdown-menu li {
    margin: 10px 0;
}
```

Note: A hover-only dropdown is not sufficient for every user. Accessible menus should also support keyboard navigation and touch devices.

---

18. CSS FILTERS

The `filter` property applies visual effects to images and other elements.

Common filter functions

Function	Purpose
`blur()`	Blurs an element
`brightness()`	Adjusts brightness
`contrast()`	Adjusts contrast
`grayscale()`	Converts an element to grayscale
`sepia()`	Adds a brownish vintage effect
`invert()`	Inverts colours
`saturate()`	Adjusts colour intensity
`opacity()`	Adjusts transparency

Examples

```
img {
    filter: grayscale(100%);
}
```

Converts an image to black and white.

```
img {
    filter: blur(3px);
}
```

Blurs an image.

```
img {
    filter: brightness(150%);
}
```

Increases image brightness.

Combining Filters

```
img {
    filter: grayscale(50%) contrast(120%) brightness(110%);
}
```

---

19. OBJECT-FIT AND OBJECT-POSITION

These properties control how images and videos fit inside a defined container.

A. object-fit

The `object-fit` property determines how replaced content, such as an image or video, fits inside its container.

Common values:

- `fill`
- `contain`
- `cover`
- `none`
- `scale-down`

Example:

```
img {
    width: 300px;
    height: 200px;
    object-fit: cover;
}
```

The image fills the container while maintaining its aspect ratio. Some parts may be cropped.

B. object-position

Controls which part of the image is displayed when cropping occurs.

```
img {
    width: 300px;
    height: 200px;
    object-fit: cover;
    object-position: center top;
}
```

This positions the visible image content towards the top centre.

Application: These properties are useful when creating profile pictures, product images, image galleries and website banners.

---

20. ADDITIONAL CSS MEASUREMENT UNITS

CSS uses different units to determine dimensions, spacing and text sizes.

Common units

Unit	Meaning
`px`	Pixels
`%`	Percentage of a reference size
`em`	Relative to the element's font size, with context-dependent inheritance
`rem`	Relative to the root element's font size
`vw`	1% of viewport width
`vh`	1% of viewport height
`vmin`	1% of the smaller viewport dimension
`vmax`	1% of the larger viewport dimension
`ch`	Based on the width of the `0` character in the relevant font
`ex`	Based on the font's x-height
`dvh`	1% of the dynamic viewport height

Examples

```
h1 {
    font-size: 5vw;
}

.container {
    width: 80%;
```

```
}

p {
    font-size: 1rem;
}

section {
    min-height: 100dvh;
}
```

Important: Use relative units where appropriate to make websites more flexible across different screen sizes and user settings.

---

21. CSS ORGANISATION AND NAMING CONVENTIONS

As websites grow, CSS files can become difficult to manage. Good organisation makes code easier to understand, update and troubleshoot.

A. Group Related Styles

```
/* General styles */
body {
    margin: 0;
    font-family: Arial;
}

/* Header styles */
header {
    background-color: green;
}

/* Navigation styles */
nav {
    padding: 15px;
}

/* Footer styles */
footer {
    background-color: black;
}
```

B. Use Meaningful Class Names

Good examples:

```
.student-card {}
.main-navigation {}
.login-form {}
```

```
.course-container {}
```

Avoid unclear names such as:

```
.x1 {}
.abc {}
.box2 {}
```

C. Use a Consistent Naming Convention

One common approach is BEM (Block, Element, Modifier).

Example:

```
.card {}
.card__title {}
.card__description {}
.card--featured {}
```

- Block: `.card`
- Element: `.card__title`
- Modifier: `.card--featured`

D. Separate CSS Files

A project can use separate files for different purposes:

```
website/
│
├── index.html
├── about.html
├── courses.html
│
├── css/
│   ├── style.css
│   ├── responsive.css
│   └── forms.css
│
└── images/
    ├── logo.png
    └── banner.jpg
```

HTML can link to the main stylesheet using:

```
<link rel="stylesheet" href="css/style.css">
```

---

22. CSS DEBUGGING USING BROWSER DEVELOPER TOOLS

Browser Developer Tools help developers inspect and troubleshoot HTML and CSS.

How to Open Developer Tools

In most browsers:

- Press `F12`.
- Press `Ctrl + Shift + I`.
- Right-click an element and select **Inspect**.

What You Can Do

1. Inspect HTML elements.
2. View applied CSS rules.
3. Identify overridden styles.
4. Change CSS values temporarily.
5. Check element dimensions.
6. Inspect responsive layouts.
7. View console errors.
8. Test different screen sizes.

Practical Example

Suppose a paragraph is not changing colour as expected.

```
p {
    color: red;
}
```

Using Developer Tools:

1. Right-click the paragraph.
2. Select Inspect.
3. Locate the Styles panel.
4. Check whether the rule is applied.
5. Look for crossed-out declarations.
6. Identify any rule overriding the colour.
7. Temporarily change the value to test your solution.

Note: Changes made directly in Developer Tools are generally temporary and do not automatically update your original CSS file.

---

23. MODERN CSS DESIGN TECHNIQUES

These techniques are commonly used in modern websites and user interfaces.

A. Glassmorphism

Glassmorphism creates a frosted-glass appearance using transparency and background blur.

```
.card {
    background: rgba(255, 255, 255, 0.2);
    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.3);
    border-radius: 15px;
    padding: 25px;
}
```

B. Smooth Scrolling

```
html {
    scroll-behavior: smooth;
}
```

This makes navigation to page sections scroll smoothly.

C. Loading Spinner

```
<div class="loader"></div>
.loader {
    width: 50px;
    height: 50px;
    border: 5px solid lightgray;
    border-top: 5px solid green;
    border-radius: 50%;
    animation: spin 1s linear infinite;
}

@keyframes spin {
    to {
        transform: rotate(360deg);
    }
}
```

D. Responsive Images

```
img {
    max-width: 100%;
    height: auto;
}
```

This prevents images from overflowing their containers.

E. Dark Mode with CSS Variables

```
:root {
    --background: white;
    --text-color: black;
}

body {
    background-color: var(--background);
    color: var(--text-color);
}

.dark-mode {
    --background: #222;
    --text-color: white;
}
```

The `.dark-mode` class can be added to the body using JavaScript or another suitable mechanism to activate the dark theme.

---

24. ACCESSIBILITY AND USER EXPERIENCE IN CSS

CSS should not only make a website attractive; it should also make it usable by different people and devices.

Important practices:

1. Use sufficient colour contrast between text and backgrounds.
2. Do not remove keyboard focus indicators without providing an accessible replacement.
3. Make buttons and links easy to identify.
4. Avoid relying on colour alone to communicate important information.
5. Use readable font sizes and appropriate line spacing.
6. Support different screen sizes.
7. Respect reduced-motion preferences for users who prefer less animation.

Example:

```
button:focus-visible,
a:focus-visible {
    outline: 3px solid orange;
    outline-offset: 3px;
}
```

Respecting reduced motion:

```
@media (prefers-reduced-motion: reduce) {
    *,
    *::before,
    *::after {
        animation-duration: 0.01ms !important;
```

```
        animation-iteration-count: 1 !important;
        scroll-behavior: auto !important;
        transition-duration: 0.01ms !important;
    }
}
```

PRACTICAL CSS EXERCISES

Exercise 1: Pseudo-classes

Create a webpage containing five links and three buttons.

Requirements:

1. Give the links different colours.
2. Change their colours when hovered over.
3. Change the button background when clicked.
4. Give input fields a different border when focused.

Exercise 2: CSS Gradients

Create a webpage with:

1. A linear gradient background.
2. A radial gradient box.
3. A conic gradient circle.
4. A heading with a text shadow.

Exercise 3: CSS Animation

Create a webpage with:

1. A bouncing ball.
2. A rotating square.
3. A colour-changing box.
4. A loading spinner.

Exercise 4: CSS Navigation

Create a navigation bar containing:

1. Home.
2. About Us.

3. Courses.
4. Learning Resources.
5. Contact Us.

Requirements:

- Use a horizontal layout.
- Add hover effects.
- Create a dropdown menu under Learning Resources.
- Make the navigation responsive.

Exercise 5: CSS Grid and Flexbox

Create a college homepage layout containing:

1. A header.
2. A navigation bar.
3. A sidebar.
4. A main content area.
5. Three course cards.
6. A footer.

Use CSS Grid for the overall page layout and Flexbox for the course cards.

Exercise 6: Modern Student Registration Form

Create a student registration form with:

- Full name.
- Email address.
- Telephone number.
- Date of birth.
- Course selection.
- Gender.
- Address.
- Submit button.

Use CSS to:

1. Add a background gradient.
2. Centre the form.
3. Add rounded corners.
4. Add box shadows.
5. Style the input fields.
6. Add focus effects.

7. Make the form responsive.

END OF CSS NOTES – PART TWO

Recommended learning progression:

1. Master selectors, specificity and inheritance.
2. Learn pseudo-classes, pseudo-elements and attribute selectors.
3. Practise gradients, shadows, filters and transforms.
4. Learn animations and CSS variables.
5. Strengthen Flexbox and Grid skills.
6. Build navigation bars, forms and responsive layouts.
7. Learn debugging, accessibility and modern design techniques.

Final learning objective: By the end of these two sets of CSS notes, students should be able to style HTML documents, control layouts, create responsive webpages, apply animations and develop modern website interfaces using CSS.

CSS NOTES – PART THREE: PROFESSIONAL CSS DEVELOPMENT

1. CSS ARCHITECTURE AND STYLESHEET MANAGEMENT

CSS architecture refers to how CSS code is structured, organised and maintained in a website project.

As websites become larger, writing all CSS rules in one place can make the stylesheet difficult to manage.

A. Organising CSS into Sections

A stylesheet can be divided into logical sections.

```
/* 1. CSS Variables */
:root {
    --primary-color: green;
    --secondary-color: orange;
    --font-main: Arial, sans-serif;
}

/* 2. General Styles */
```

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: var(--font-main);
}

/* 3. Header */
header {
    background-color: var(--primary-color);
}

/* 4. Navigation */
nav {
    padding: 15px;
}

/* 5. Main Content */
main {
    padding: 20px;
}

/* 6. Footer */
footer {
    background-color: black;
    color: white;
}
```

B. CSS Reset

Different browsers may apply different default margins, padding and styles to HTML elements.

A CSS reset helps create a more predictable starting point.

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

A more selective reset can also be used:

```
html {
    box-sizing: border-box;
}

*, *::before, *::after {
    box-sizing: inherit;
}

body {
    margin: 0;
```

```
}
```

C. External Stylesheets

A website can use multiple CSS files for different purposes.

Example folder structure:

```
college-website/
│
├── index.html
├── about.html
│
├── css/
│   ├── reset.css
│   ├── variables.css
│   ├── layout.css
│   ├── components.css
│   └── responsive.css
│
└── images/
    └── logo.png
```

Linking multiple stylesheets:

```
<link rel="stylesheet" href="css/reset.css">
<link rel="stylesheet" href="css/variables.css">
<link rel="stylesheet" href="css/layout.css">
<link rel="stylesheet" href="css/components.css">
<link rel="stylesheet" href="css/responsive.css">
```

The order matters because later declarations can override earlier declarations when specificity and other cascade conditions are equal.

---

2. CSS RESPONSIVE DESIGN – ADVANCED CONCEPTS

Responsive design allows websites to adjust their appearance and layout according to the device or screen size.

A. Viewport Meta Tag

Every responsive HTML webpage should normally include:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

This helps mobile browsers use the device's actual viewport width.

B. Media Queries

Media queries apply CSS rules depending on device or viewport characteristics.

Syntax:

```
@media condition {
    /* CSS rules */
}
```

Example:

```
body {
    background-color: white;
}

@media screen and (max-width: 600px) {
    body {
        background-color: lightblue;
    }
}
```

The background changes when the viewport is 600px wide or narrower.

C. Multiple Breakpoints

```
/* Small devices */
@media (max-width: 600px) {
    .container {
        width: 100%;
    }
}

/* Medium devices */
@media (min-width: 601px) and (max-width: 900px) {
    .container {
        width: 90%;
    }
}

/* Large devices */
@media (min-width: 901px) {
    .container {
        width: 80%;
    }
}
```

D. Mobile-First Design

Mobile-first design means developing the mobile layout first, then adding styles for larger screens.

```
.container {
    display: flex;
    flex-direction: column;
}

@media (min-width: 768px) {
    .container {
        flex-direction: row;
    }
}
```

On small screens, items appear vertically. On larger screens, they appear horizontally.

E. Responsive Typography

The `clamp()` function allows text to resize within defined limits.

```
h1 {
    font-size: clamp(25px, 4vw, 55px);
}
```

F. Responsive Images

```
img {
    max-width: 100%;
    height: auto;
}
```

This prevents images from extending beyond their containers.

G. Responsive Navigation

A navigation menu can change from horizontal to vertical on smaller screens.

```
nav ul {
    display: flex;
    list-style: none;
    gap: 20px;
}

@media (max-width: 600px) {
    nav ul {
        flex-direction: column;
    }
}
```

Note: CSS can rearrange the navigation, but a functional hamburger menu that opens and closes on click normally requires JavaScript or another interactive mechanism.

---

3. CSS CONTAINER QUERIES

Container queries allow CSS styles to depend on the size of a parent container rather than the entire browser window.

This is particularly useful when the same component appears in different parts of a website.

A. Declaring a Container

```
.card-container {
    container-type: inline-size;
}
```

B. Applying a Container Query

```
@container (min-width: 500px) {
    .card {
        display: flex;
        gap: 20px;
    }
}
```

When the container reaches 500px or wider, the card changes to a horizontal layout.

C. Practical Example

HTML:

```
<div class="card-container">
    <div class="course-card">
        <img src="course.jpg" alt="Computer course">
        <div>
            <h2>Web Development</h2>
            <p>Learn modern website development.</p>
        </div>
    </div>
</div>
```

CSS:

```
.card-container {
    container-type: inline-size;
}

.course-card {
    display: block;
    padding: 15px;
    border: 1px solid gray;
}

.course-card img {
```

```
    width: 100%;
}

@container (min-width: 500px) {
    .course-card {
        display: flex;
        align-items: center;
        gap: 20px;
    }

    .course-card img {
        width: 180px;
    }
}
```

Difference: Media queries respond to viewport or device conditions, while container queries respond to the size of a designated container.

---

4. CSS SUBGRID

Subgrid is an advanced CSS Grid feature that allows a nested grid to use the row or column tracks of its parent grid.

It is useful when several cards need their content aligned consistently.

Example:

```
.container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}

.card {
    display: grid;
    grid-template-rows: subgrid;
    grid-row: span 3;
}
```

Subgrid is particularly useful in complex card layouts and content sections where items need to align across different containers.

---

5. CSS MULTI-COLUMN LAYOUT

CSS Multi-column Layout allows text to flow into multiple columns, similar to a newspaper or magazine.

A. column-count

Specifies the number of columns.

```
.article {
    column-count: 3;
}
```

B. column-gap

Controls the space between columns.

```
.article {
    column-gap: 30px;
}
```

C. column-rule

Adds a line between columns.

```
.article {
    column-rule: 1px solid gray;
}
```

D. Complete Example

```
.article {
    column-count: 3;
    column-gap: 30px;
    column-rule: 1px solid lightgray;
}
```

HTML:

```
<div class="article">
    <p>
        CSS is used to style HTML documents.
        It controls colours, layouts, fonts,
        spacing and many other visual properties.
    </p>
</div>
```

---

6. CSS WRITING MODES AND TEXT DIRECTION

CSS writing modes control the direction in which text and other content are arranged.

A. Writing Mode

```
.vertical-text {
    writing-mode: vertical-rl;
}
```

This displays text vertically from top to bottom, with lines progressing from right to left.

Common values:

- `horizontal-tb`
- `vertical-rl`
- `vertical-lr`

B. Direction

The `direction` property determines the direction of text.

```
.text {
    direction: rtl;
}
```

Values:

- `ltr` – Left to right.
- `rtl` – Right to left.

C. Text Alignment

```
p {
    text-align: justify;
}
```

Other common values include:

- `left`
- `right`
- `center`
- `justify`

These properties are useful when working with multilingual websites and different writing systems.

---

7. CSS LOGICAL PROPERTIES

Logical properties allow developers to control spacing and positioning according to the writing direction and writing mode.

They are useful when developing websites that support both left-to-right and right-to-left languages.

A. Traditional Properties

```
.box {
    margin-left: 20px;
    padding-right: 15px;
}
```

B. Logical Properties

```
.box {
    margin-inline-start: 20px;
    padding-inline-end: 15px;
}
```

Common Logical Properties

Logical property	General purpose
`margin-inline-start`	Margin at the start of the inline direction
`margin-inline-end`	Margin at the end of the inline direction
`padding-block-start`	Padding at the beginning of the block direction
`padding-block-end`	Padding at the end of the block direction
`border-inline-start`	Border at the start of the inline direction
`inset-inline-start`	Positioning from the start of the inline direction
`inline-size`	Size along the inline direction
`block-size`	Size along the block direction

Example:

```
.card {
    padding-inline: 20px;
    padding-block: 15px;
}
```

This applies padding along the inline and block directions.

---

8. CSS SCROLL BEHAVIOUR AND SCROLL SNAPPING

CSS provides properties that control scrolling and how elements align when scrolling.

A. Smooth Scrolling

```
html {
    scroll-behavior: smooth;
}
```

B. Scroll Snap

Scroll snapping allows scrolling containers to stop at specific positions.

```
.gallery {
    display: flex;
    overflow-x: auto;
    scroll-snap-type: x mandatory;
    gap: 15px;
}

.gallery-item {
    flex: 0 0 250px;
    scroll-snap-align: start;
}
```

HTML:

```
<div class="gallery">
    <div class="gallery-item">Image One</div>
    <div class="gallery-item">Image Two</div>
    <div class="gallery-item">Image Three</div>
</div>
```

C. Scroll Margin

```
section {
    scroll-margin-top: 80px;
}
```

This provides extra spacing when scrolling to a section, useful when a website has a fixed navigation bar.

---

9. CSS BLEND MODES

Blend modes determine how an element's colours interact with the colours behind it.

A. mix-blend-mode

Controls how an element blends with the content behind it.

```
.image {
    mix-blend-mode: multiply;
}
```

Common values include:

- `normal`
- `multiply`
- `screen`
- `overlay`
- `darken`
- `lighten`
- `difference`
- `exclusion`

B. background-blend-mode

Controls how multiple backgrounds blend with each other.

```
.banner {
    background-image:
        linear-gradient(rgba(0,0,0,0.5), rgba(0,0,0,0.5)),
        url("banner.jpg");

    background-blend-mode: multiply;
}
```

This is useful when placing text over images while improving text readability.

---

10. CSS BACKDROP FILTER

The `backdrop-filter` property applies visual effects to the area behind an element.

Unlike `filter`, which affects the element itself, `backdrop-filter` affects the content behind the element.

Example:

```
.glass {
    background: rgba(255,255,255,0.2);
    backdrop-filter: blur(10px);
    padding: 30px;
}
```

Common effects include:

- `blur()`
- `brightness()`
- `contrast()`
- `grayscale()`

- `saturate()`

Application: Glass-style navigation bars, floating panels and modern login screens.

---

11. CSS CLIPPING AND MASKING

These techniques control which parts of an element are visible.

A. clip-path

The `clip-path` property creates a clipping region that determines which parts of an element remain visible.

Example:

```
.image {
    clip-path: circle(50%);
}
```

This clips an image into a circle.

B. Polygon Shapes

```
.box {
    clip-path: polygon(
        50% 0,
        100% 100%,
        0 100%
    );
}
```

This creates a triangular visible area.

C. ellipse()

```
.image {
    clip-path: ellipse(40% 50%);
}
```

This clips an element into an elliptical shape.

D. CSS Masking

The `mask-image` property uses an image or gradient as a mask to control the visibility of an element.

```
.image {
    mask-image: linear-gradient(black, transparent);
}
```

Masking is useful for fading images and creating decorative effects.

---

12. CSS BLUR AND FILTER EFFECTS ON BACKGROUNDS

A common design requirement is to blur a background image while keeping the text in front clear.

One approach is to use a pseudo-element.

HTML:

```
<div class="hero">
    <div class="hero-content">
        <h1>Welcome to RuralNet</h1>
        <p>Bridging the digital divide.</p>
    </div>
</div>
```

CSS:

```
.hero {
    position: relative;
    overflow: hidden;
    padding: 100px 20px;
}

.hero::before {
    content: "";
    position: absolute;
    inset: 0;
    background: url("background.jpg") center/cover;
    filter: blur(5px);
    transform: scale(1.05);
}

.hero-content {
    position: relative;
    z-index: 1;
    color: white;
    text-align: center;
}
```

The background is blurred while the text remains clear.

---

13. CSS PRINT STYLESHEETS

CSS can control how a webpage appears when printed.

This is useful when creating printable reports, invoices, examination papers and registration forms.

A. Print Media Query

```
@media print {
    body {
        background: white;
        color: black;
    }

    nav,
    button,
    .no-print {
        display: none;
    }
}
```

The navigation and buttons will not appear in printed output.

B. Page Size

```
@page {
    size: A4;
    margin: 20mm;
}
```

C. Avoid Breaking Important Content

```
.card {
    break-inside: avoid;
}
```

This helps prevent a card or other content block from splitting across printed pages.

Application: Printing student records, fee statements, receipts and academic reports.

---

14. CSS SUPPORTS AND FEATURE DETECTION

Different browsers may support different CSS features.

The `@supports` rule allows CSS to apply styles only when the browser supports a particular property or value.

Example:

```
@supports (display: grid) {
    .container {
        display: grid;
    }
}
```

Another example:

```
@supports (backdrop-filter: blur(10px)) {
    .glass {
        backdrop-filter: blur(10px);
    }
}
```

A fallback can be provided:

```
.glass {
    background: white;
}

@supports (backdrop-filter: blur(10px)) {
    .glass {
        background: rgba(255,255,255,0.3);
        backdrop-filter: blur(10px);
    }
}
```

This ensures that the design remains usable when a browser does not support the advanced effect.

---

15. CSS PREFERS-REDUCED-MOTION

Some users prefer websites with fewer animations and movement effects.

CSS can detect this preference.

```
@media (prefers-reduced-motion: reduce) {
    * {
        animation-duration: 0.01ms !important;
        transition-duration: 0.01ms !important;
        scroll-behavior: auto !important;
    }
}
```

This helps make websites more accessible.

---

16. CSS PERFORMANCE OPTIMISATION

CSS performance optimisation involves writing styles that load and render efficiently.

Important Practices

1. Remove unused CSS.
2. Avoid unnecessarily complicated selectors.
3. Reuse classes instead of repeating declarations.
4. Minimise stylesheet files for production where appropriate.
5. Avoid excessive animations.
6. Use modern layout techniques.
7. Avoid unnecessary large shadows and filters.
8. Test website performance on mobile devices.

A. CSS Minification

Minification removes unnecessary spaces, comments and line breaks from CSS code.

Normal CSS:

```
body {
    background-color: white;
    color: black;
}
```

Minified CSS:

```
body{background-color:white;color:black}
```

Minification reduces file size but makes the code less readable for manual editing.

B. Avoid Unnecessary Universal Selectors

Avoid applying expensive or unnecessary styling to every element.

Instead of:

```
* {
    box-shadow: 0 0 10px gray;
}
```

Apply the shadow only to the elements that need it:

```
.card {
    box-shadow: 0 0 10px gray;
}
```

C. Prefer Transform and Opacity for Many Animations

For many movement and fading effects, `transform` and `opacity` are preferable to repeatedly animating layout properties such as width, height or margin.

Example:

```
.card {
    transition: transform 0.3s ease;
}

.card:hover {
    transform: translateY(-5px);
}
```

---

17. CSS PREPROCESSORS

A CSS preprocessor is a tool that extends ordinary CSS with additional features and generates standard CSS that browsers can understand.

Popular preprocessors include:

1. Sass/SCSS.
2. Less.
3. Stylus.

A. Sass/SCSS

SCSS supports variables, nesting, mixins and functions.

Example:

```
$primary-color: green;

body {
    font-family: Arial;

    h1 {
        color: $primary-color;
    }
}
```

The SCSS compiler converts this into ordinary CSS.

B. Nesting

SCSS allows selectors to be nested.

```
nav {
    background: green;

    ul {
        list-style: none;
    }

    a {
        color: white;
    }
}
```

C. Mixins

Mixins allow reusable groups of CSS declarations.

```
@mixin button-style {
    padding: 12px 20px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

button {
    @include button-style;
}
```

Note: Modern CSS also supports native nesting and several features that were traditionally associated with preprocessors. Sass remains useful for larger projects and established workflows.

---

18. CSS FRAMEWORKS AND LIBRARIES

CSS frameworks provide predefined styles and layout systems that help developers create websites faster.

Common CSS Frameworks

A. Bootstrap

Bootstrap provides:

- Grid system.
- Buttons.

- Navigation bars.
- Cards.
- Forms.
- Modals.
- Responsive utilities.

Website: https://getbootstrap.com/

B. Tailwind CSS

Tailwind CSS uses utility classes that are applied directly in HTML.

Example:

```
<div class="bg-green-600 text-white p-5 rounded-lg">
    Welcome to RuralNet
</div>
```

Website: https://tailwindcss.com/

C. Bulma

Bulma provides a CSS-based framework with ready-made components and responsive layouts.

Website: https://bulma.io/

D. Materialize CSS

Materialize CSS provides components and design styles based on Material Design principles.

Website: https://materializecss.com/

Advantages of CSS Frameworks

1. Faster development.
2. Reusable components.
3. Responsive layouts.
4. Consistent design.
5. Less need to write common CSS from scratch.

Disadvantages

1. Some frameworks add unnecessary code.
2. Websites may look similar if not customised.
3. Students may become dependent on predefined classes.
4. Learning the underlying CSS can be neglected.

Recommendation: Students should understand ordinary CSS before relying heavily on frameworks.

---

19. CSS CROSS-BROWSER COMPATIBILITY

Cross-browser compatibility means ensuring that a website works consistently across different browsers.

Examples include:

- Google Chrome.
- Mozilla Firefox.
- Microsoft Edge.
- Safari.
- Opera.

Common Challenges

1. Different default styles.
2. Different support for newer CSS features.
3. Font rendering differences.
4. Form control appearance differences.
5. Differences in viewport handling.

Good Practices

1. Use standard CSS properties.
2. Test in more than one browser.
3. Provide fallbacks for advanced features.
4. Use feature detection where appropriate.
5. Avoid depending on experimental properties without a fallback.

Vendor Prefixes

Some CSS features historically required browser-specific prefixes.

Examples:

```
.example {
    -webkit-backdrop-filter: blur(10px);
    backdrop-filter: blur(10px);
}
```

Prefixes should only be added when required by the browser support targets of the project. Modern development tools can help manage them.

---

20. CSS DEVELOPER TOOLS AND WORKFLOW

Professional developers use browser tools and other development utilities to inspect, test and improve CSS.

A. Browser Inspector

Use the browser's Inspect tool to:

- View HTML structure.
- Inspect applied CSS.
- Identify conflicting rules.
- Change values temporarily.
- Inspect element dimensions.

B. Responsive Design Mode

Use Developer Tools to test different viewport sizes.

Common test sizes:

- 375px – Small mobile screen.
- 768px – Tablet-sized screen.
- 1024px – Small desktop or tablet landscape.
- 1440px – Large desktop screen.

These are example testing widths, not fixed device standards.

C. CSS Validation

CSS validators can help identify syntax problems.

Website: https://jigsaw.w3.org/css-validator/

D. Browser Compatibility Checking

Website: https://caniuse.com/

This helps developers check browser support for CSS features.

PRACTICAL PROJECTS

PROJECT 1: RESPONSIVE COLLEGE WEBSITE

Create a college website containing:

1. Header with a college logo.
2. Navigation bar.
3. Hero section with a background image.
4. About section.
5. Courses section.
6. Three course cards.
7. Contact form.
8. Footer.

Requirements:

- Use CSS Grid and Flexbox.
- Add responsive breakpoints.
- Use CSS variables.
- Add hover effects.
- Use gradients and shadows.
- Make images responsive.
- Test on mobile and desktop.

PROJECT 2: STUDENT DASHBOARD

Create a student dashboard with:

1. Sidebar navigation.
2. Top navigation bar.
3. Student profile section.
4. Course information cards.
5. Fee balance section.
6. Examination results section.
7. Footer.

Requirements:

- Use CSS Grid for the main layout.
- Use Flexbox for individual components.
- Add responsive behaviour.

- Use CSS variables.
- Add hover and focus effects.
- Include a dark-mode design using CSS variables.

PROJECT 3: PRINTABLE STUDENT REPORT

Create a student academic report containing:

1. College name and logo.
2. Student information.
3. Course information.
4. Examination results table.
5. Performance summary.
6. Signature section.

Requirements:

- Use CSS for professional document styling.
- Use `@media print`.
- Use `@page` to configure the printed page.
- Hide unnecessary website elements when printing.
- Ensure tables and important content print correctly.

FINAL CSS LEARNING OBJECTIVES

By the end of all three sets of CSS notes, students should be able to:

1. Explain the purpose and syntax of CSS.
2. Apply CSS using inline, internal and external stylesheets.
3. Use different CSS selectors.
4. Understand specificity, inheritance and the cascade.
5. Style text, images, links, tables and forms.
6. Understand the CSS box model.
7. Create layouts using Flexbox and Grid.
8. Develop responsive webpages.
9. Create transitions and animations.
10. Use CSS variables and functions.
11. Apply modern design techniques.
12. Use browser Developer Tools to debug CSS.
13. Prepare webpages for printing.
14. Understand CSS frameworks and preprocessors.
15. Apply accessibility and performance principles.
16. Develop professional websites using HTML and CSS.

END OF CSS NOTES – PART THREE